arXiv:2607.05147v1 [cs.AI] 6 Jul 2026

DSpark: Confidence-Scheduled Speculative Decoding with Semi-Autoregressive Generation

Xin Cheng[1,2,*], Xingkai Yu[2,*], Chenze Shao[2,*], Jiashi Li[2,*], Yunfan Xiong[2,*]
Yi Qian[2], Jiaqi Zhu[2], Shirong Ma[2], Xiaokang Zhang[2], Jiasheng Ye[2], Qinyu Chen[2],
Chengqi Deng[2], Jiping Yu[2], Damai Dai[2], Zhengyan Zhang[2], Yixuan Wei[2], Yixuan Tan[2],
Wenkai Yang[2], Runxin Xu[2], Yu Wu[2], Zhean Xu[2], Xuanyu Wang[2], Muyang Chen[2],
Rui Tian[2], Xiao Bi[2], Zhewen Hao[2], Shaoyuan Chen[2], Huanqi Cao[2], Wentao Zhang[2],
Anyi Xu[2], Huishuai Zhang[1], Dongyan Zhao[1], Wenfeng Liang[2]

[1]**Peking University** [2]**DeepSeek-AI**

{chengxin, xingkai, shaochenze, js.li, yunfanxiong}@deepseek.com

Abstract

Speculative decoding accelerates Large Language Model (LLM) inference by decoupling draft generation from target verification. While recent parallel drafters efficiently propose long token sequences in a single forward pass, they suffer from rapid acceptance decay due to a lack of inter-token dependencies. Furthermore, indiscriminately verifying these extended blocks wastes critical batch capacity on tokens with high rejection risks, severely degrading throughput in high-concurrency serving systems. We introduce DSpark, a speculative decoding framework that unifies high-throughput parallel generation with adaptive, load-aware verification. To maintain draft quality, DSpark utilizes a semi-autoregressive architecture—coupling a parallel backbone with a lightweight sequential module—to introduce intra-block dependency modeling and mitigate suffix decay. To optimize system efficiency, DSpark employs confidence-scheduled verification, dynamically tailoring the verification length for each request based on estimated prefix survival probabilities and engine-specific throughput profiles. On offline benchmarks across diverse domains, DSpark substantially improves the accepted length over state-of-the-art autoregressive and parallel drafters. When deployed within the DeepSeek-V4 serving system under live user traffic, DSpark successfully mitigates verification waste. Compared to the established production baseline (MTP-1), DSpark accelerates per-user generation speeds by 60%–85% at matched throughput levels. More importantly, by preventing severe throughput degradation under strict interactivity constraints, it enables performance tiers that were previously unattainable, shifting the Pareto frontier of our serving system. To facilitate community progress, we open-source the DSpark checkpoints alongside DeepSpec, an algorithm-driven training repository for speculative decoding.

1. Introduction

Large Language Models (LLMs) generate text autoregressively: each new token requires a full forward pass conditioned on all preceding tokens, making inference latency proportional to the output length. The resulting low GPU utilization and high user-perceived waiting time constitute

[*] Equal contribution.

a primary bottleneck in production LLM serving, particularly for latency-sensitive scenarios such as real-time conversational assistants and multi-turn agentic workflows. Speculative decoding (Chen et al., 2023; Leviathan et al., 2023) offers a principled solution: a lightweight *draft model* proposes a block of candidate tokens, and the full-size *target model* verifies the entire block in a single forward pass via rejection sampling, accepting the longest prefix consistent with the target distribution and appending one bonus token. Because verification is parallel and the acceptance rule preserves the target distribution exactly, speculative decoding accelerates generation without any quality loss.

The design of the draft model governs the trade-off between drafting latency and acceptance rate. Early drafters are autoregressive (Cheng et al., 2024; Li et al., 2024b), conditioning each position on previously sampled tokens. However, their drafting latency grows linearly with the block size, forcing these methods to use short blocks and shallow architectures. To break this sequential bottleneck, parallel drafters (Cai et al., 2024; Chen et al., 2026; Liu et al., 2026a) have emerged as a compelling alternative: all draft positions are produced in a single forward pass, making drafting latency nearly independent of block size. This structural advantage theoretically allows parallel drafters to efficiently generate substantially longer draft blocks.

However, fully unlocking the potential of large parallel draft blocks introduces two critical bottlenecks—one in generation quality, and the other in system efficiency. First, because parallel drafters predict each position independently, they cannot model inter-token dependencies within a block. This independence leads to multi-modal collisions and rapid acceptance decay at later positions (Gu et al., 2018; Huang et al., 2022b). Second, determining the optimal verification length remains a challenge. While parallel generation easily produces long draft blocks, indiscriminately verifying all proposed tokens degrades system throughput, particularly under high-concurrency workloads (Hu et al., 2026b; Liu et al., 2024c). The ideal verification length varies along two axes. On the data side, structured requests like code naturally sustain higher acceptance rates than open-ended chat (Abramovich et al., 2026; Xia et al., 2024). On the system side, verifying extra tokens is nearly free under light loads. Under heavy loads, however, verifying tokens with a high rejection risk occupies critical batch capacity that could otherwise serve other active requests (Liu et al., 2024b; Wu et al., 2025).

To address these bottlenecks, we introduce **DSpark**, a speculative decoding framework that unifies high-throughput parallel generation with adaptive, load-aware verification. At its core, DSpark is designed to resolve the inherent trade-offs in draft generation and verification through two complementary mechanisms.

- First, to overcome the lack of inter-token dependencies, DSpark adopts a semi-autoregressive architecture. It keeps the computationally expensive draft backbone fully parallel, appending only a lightweight serial output head to inject local transition information. This design preserves the drafting speed of parallel models while significantly mitigating suffix decay.
- Second, to resolve the system-level bottleneck, DSpark employs confidence-scheduled verification. By coupling a confidence head—which estimates per-position prefix survival probabilities—with a hardware-aware scheduler, DSpark dynamically tailors the verification length for each request. This scheduler leverages real-time engine throughput profiles to route target verification budget only toward tokens with the highest expected return.

We extensively evaluate DSpark across both controlled offline benchmarks and production-scale online deployments. On controlled offline benchmarks—spanning mathematical reasoning, code generation, and daily chat—DSpark consistently outperforms strong baselines. Specifically,

across the Qwen3-4B, 8B, and 14B target models (Yang et al., 2025), it improves the macro-average accepted length over the autoregressive Eagle3 (Li et al., 2026b) by 30.9%, 26.7%, and 30.0%, and over the parallel DFlash (Chen et al., 2026) by 16.3%, 18.4%, and 18.3%, respectively. Beyond top-line metrics, our fine-grained position-wise analysis reveals the distinct generation characteristics of different drafters, empirically demonstrating how DSpark successfully combines the high initial-token capacity of parallel models with the suffix coherence of autoregressive models.

Beyond offline evaluation, we deployed DSpark within the DeepSeek-V4 (DeepSeek-AI, 2026) serving system to assess its performance under live user traffic. Compared to the prior MTP-1 production baseline (DeepSeek-AI, 2024), DSpark significantly broadens the system's operational envelope. Specifically, it consistently accelerates per-user generation speeds by 60%–85% (V4-Flash) and 57%–78% (V4-Pro) at matched aggregate throughput capacities. Furthermore, under strict Service Level Agreements (SLAs) where the baseline's capacity deteriorates severely—such as 120 TPS for Flash and 50 TPS for Pro—DSpark mitigates verification overhead to maintain robust throughput. By overcoming this performance cliff, DSpark unlocks strict interactivity tiers that were previously unattainable, effectively shifting the Pareto frontier of LLM serving.

To foster collective advancement within the open-source community, we are making our artifacts publicly available. Specifically, we release the trained DSpark checkpoints for both the DeepSeek-V4-Flash (preview) and DeepSeek-V4-Pro (preview) models. Furthermore, we open-source DeepSpec, an algorithm-driven training repository, including Eagle3, DFlash and DSpark. These artifacts are intended to support further research on efficient LLM serving.

2. Background

2.1. Speculative Decoding

Autoregressive language models generate one token per forward pass, making inference latency proportional to output length. Speculative decoding (Chen et al., 2023; Ge et al., 2022; Leviathan et al., 2023) accelerates the inference of a target model M_t using a lightweight draft model M_d. At each decoding cycle, the draft model proposes γ candidate tokens $x_1, \ldots, x_\gamma$. The target model verifies all candidates in a single forward pass, accepting the longest prefix consistent with its own distribution.

Concretely, at each draft position k, the target model computes its own distribution p_k^t and compares it against the draft distribution p_k^d. The token x_k is accepted with probability $\min(1, p_k^t(x_k)/p_k^d(x_k))$. Verification proceeds left to right: the first rejection at position k discards all subsequent tokens $x_{k+1}, \ldots, x_\gamma$, regardless of their quality.

Let τ denote the number of accepted tokens per cycle, and let T_{draft} and T_{verify} be the wall-clock times of the drafting and verification passes, respectively. The average latency per generated token is:

$$L = \frac{T_{\text{draft}} + T_{\text{verify}}}{\tau}. \tag{1}$$

Improving speedup therefore reduces to three levers: lowering T_{draft} (draft faster), raising τ (draft better), or reducing the effective T_{verify} (verify smarter).

2.2. Drafter Architectures

The design of the draft model determines how T_{draft} and τ trade off. Existing approaches fall into two categories.

Autoregressive drafters. Autoregressive drafters generate draft tokens sequentially, conditioning each position on previously sampled tokens (DeepSeek-AI, 2024; Li et al., 2024b,c, 2026b; Zhang et al., 2025). This explicit dependency gives strong modeling capacity, but the drafting cost grows linearly with block size: $T_{\text{draft}} \propto \gamma$, which forces autoregressive drafters to use small γ and shallow architectures to keep T_{draft} low. To compensate for the short block, tree-based verification (Miao et al., 2024) expands candidates into a tree and verifies multiple paths via tree attention, but the large number of verification tokens reduces overall serving throughput.

Parallel drafters. Parallel drafters produce all γ draft tokens in a single forward pass, making T_{draft} nearly independent of the block size (Cai et al., 2024; Chen et al., 2026; Li et al., 2025a; Liu et al., 2026a; Sandler et al., 2026). This allows substantially larger blocks (e.g., γ=16) without proportionally increasing latency.

Among them, DFlash (Chen et al., 2026) is a state-of-the-art parallel drafter, which conditions its draft model on rich context features extracted from the target model (KV injection). During prefill, hidden states from a set of target layers $\{l_1, \ldots, l_m\}$ are concatenated and projected into the draft hidden space:

$$H_{\text{ctx}} = \text{RMSNorm}\big(W_c\,[H^{(l_1)}; \ldots; H^{(l_m)}]\big), \tag{2}$$

where $W_c \in \mathbb{R}^{d \times md}$ is a shared projection. These context features are injected into every draft layer by concatenating them with the draft block representations along the sequence dimension of keys and values:

$$K_i = [W_i^K H_{\text{ctx}};\ W_i^K H_d], \quad V_i = [W_i^V H_{\text{ctx}};\ W_i^V H_d]. \tag{3}$$

All positions within a block attend bidirectionally to each other and to the injected target context.

The draft model shares the target model's embedding layer and language modeling head (both frozen). It takes as input the embedding of an anchor token[1] followed by γ mask token embeddings, and produces logits for all mask positions in a single forward pass. Since drafting requires only a single forward pass regardless of block size, DFlash can afford deeper architectures and larger blocks than autoregressive drafters under the same latency budget.

3. Architecture

The overview of DSpark is shown in Figure 1. Recall from Equation 1 that the per-token latency of speculative decoding is $L = (T_{\text{draft}} + T_{\text{verify}})/\tau$. Autoregressive drafters achieve high τ but pay $T_{\text{draft}} \propto \gamma$; parallel drafters collapse T_{draft} to a single pass but sacrifice τ because each position is predicted independently. Meanwhile, fixed-length verification wastes T_{verify} on low-confidence suffix tokens that are almost certain to be rejected. DSpark addresses these limitations with two complementary components:

- **Semi-autoregressive generation** (Section 3.1). A parallel backbone handles the bulk of draft computation, which keeps T_{draft} nearly independent of γ. A lightweight sequential block then injects dependency among draft tokens, improving τ at minimal additional latency.
- **Confidence-scheduled verification** (Section 3.2). A confidence head estimates per-position acceptance probabilities, and a hardware-aware scheduler uses these estimates to prune low-confidence suffix tokens, cutting unnecessary verification compute.

[1] We use the terms *anchor token* and *bonus token* interchangeably in this paper to denote the final token generated by the target model in the previous decoding round.

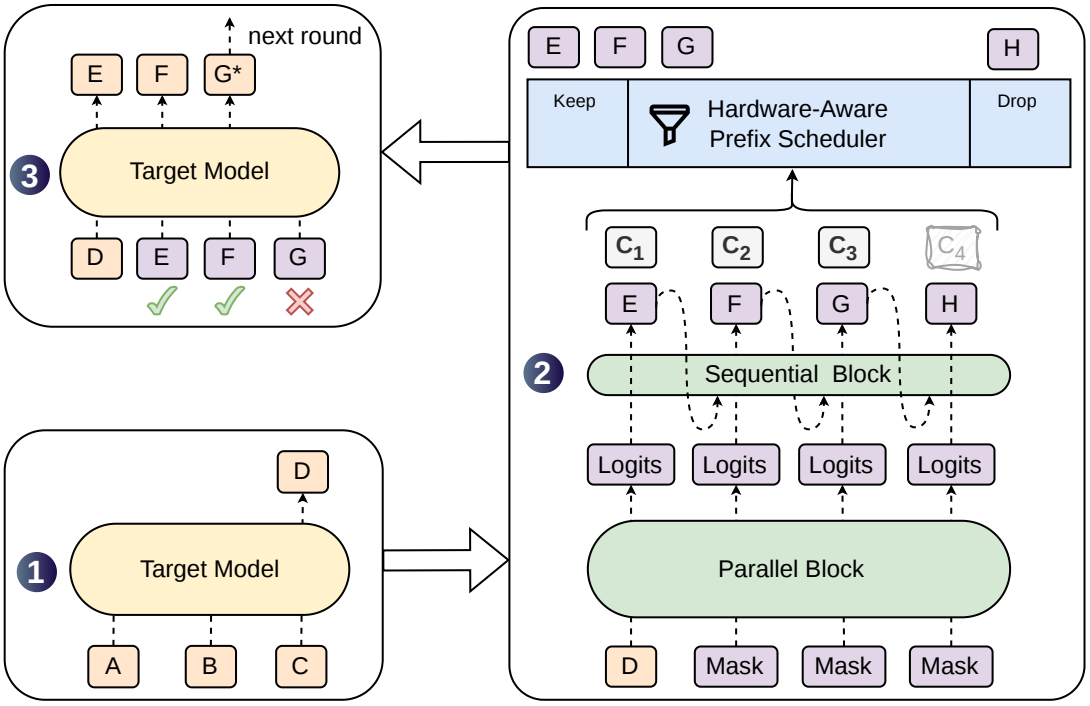

Figure 1 | **The DSpark architecture and decoding cycle.** Given prompt tokens ABC, the target model executes one step to generate the next token D, which serves as the anchor for the drafting phase. Using D as the input, DSpark employs a heavy parallel backbone and a lightweight sequential head to generate draft tokens EFGH along with their corresponding confidence scores c_1–c_4. The Hardware-Aware Prefix Scheduler then evaluates these scores to retain the prefix EFG and drop the low-confidence token H. Finally, the target model verifies the scheduled prefix in parallel. As illustrated, E and F are accepted while G is rejected, prompting the model to generate a corrected token G* to complete the current round.

3.1. Semi-Autoregressive Generation

A parallel drafter produces all γ draft logits in one forward pass, so each prediction cannot condition on tokens sampled elsewhere in the block. When the context admits multiple plausible continuations, e.g., "of course" and "no problem", a parallel drafter may produce incoherent combinations such as "of problem" or "no course", because each position marginalizes over all possible predecessors rather than conditioning on the one actually sampled (Gu et al., 2018; Huang et al., 2022a). Acceptance rate thus decays rapidly along the block, wasting both draft and verification compute. We therefore adopt a **semi-autoregressive** structure that splits draft generation into two stages:

Parallel stage. A parallel backbone (in our instantiation, DFlash (Chen et al., 2026)) runs a single forward pass over the entire block, producing hidden states $h_1, \ldots, h_\gamma$ and base logits $U_1, \ldots, U_\gamma$. We make only a minor modification to the original DFlash backbone: instead of feeding an anchor token plus γ mask tokens and predicting only the mask positions, we treat the anchor itself as the first prediction position, so γ input tokens (anchor + $\gamma-1$ masks) yield γ draft logits. This reduces draft computation while maintaining similar draft quality.

Sequential stage. The sequential stage supplements the base logits with a prefix-dependent transition bias $B_k(x_0, x_{<k}, x_k)$, allowing each draft position to condition on previously sampled tokens within the block. Rather than defining a globally normalized energy model, the sequential stage induces a causal block distribution through an autoregressive factorization:

$$P(X \mid x_0) = \prod_{k=1}^{\gamma} p_k(x_k \mid x_0, x_{<k}), \qquad p_k(v \mid x_0, x_{<k}) = \frac{\exp(U_k(v) + B_k(x_0, x_{<k}, v))}{\sum_{u \in \mathcal{V}} \exp(U_k(u) + B_k(x_0, x_{<k}, u))}. \tag{4}$$

Here, x_0 denotes the anchor token from the previous verification cycle, U_k is the base logit vector produced by the parallel backbone at position k, and $\mathcal{V}$ is the vocabulary. At inference time, the sequential block samples left to right according to $p_k(\cdot \mid x_0, x_{<k})$. Because this sampling process is inherently sequential, the block must be computationally lightweight ($T_{\text{sequential}} \ll T_{\text{parallel}}$) so that the overall draft latency remains dominated by the parallel stage. We describe two instantiations of the sequential block below.

- **Markov head.** The simplest instantiation restricts B_k to depend only on the immediately preceding token, reducing it to a first-order transition $B(x_{k-1}, x_k)$. In principle this is a full $V \times V$ matrix B; we approximate it with a low-rank factorization $B = W_1 W_2$, where $W_1 \in \mathbb{R}^{V \times r}$ and $W_2 \in \mathbb{R}^{r \times V}$. Given the preceding token x_{k-1}, the transition bias for position k is:

$$B(x_{k-1}, \cdot) = W_1[x_{k-1}]\, W_2 \ \in \ \mathbb{R}^V, \tag{5}$$

  where W_1 serves as an embedding lookup table and W_2 as a logit projection. The low-rank factorization (r=256 by default) keeps both storage and per-step compute small, making the sequential loop efficient even for large vocabularies. Returning to the earlier example: once position 1 samples "of", the Markov head boosts "course" and suppresses "problem" at position 2, which mitigates the cross-mode collision.

- **RNN head.** The Markov head is memoryless beyond one step—position k cannot access tokens before x_{k-1}. The RNN head relaxes this by maintaining a recurrent state s_k that accumulates the full prefix history within a block. At each step, the module concatenates the current state $s_{k-1} \in \mathbb{R}^r$, the previous token embedding $W_1[x_{k-1}] \in \mathbb{R}^r$, and the backbone hidden $h_k \in \mathbb{R}^d$ into an input vector $z_k = [s_{k-1}; W_1[x_{k-1}]; h_k] \in \mathbb{R}^{2r+d}$, then applies a single gated update:

$$\begin{aligned} s_k &= \sigma(W_g\, z_k) \odot s_{k-1} \ + \ \big(1 - \sigma(W_g\, z_k)\big) \odot \tanh(W_c\, z_k), \\ B_k(x_{<k}, \cdot) &= W_2^\top \ \tanh(W_o\, z_k), \end{aligned} \tag{6}$$

  where $W_g, W_c, W_o \in \mathbb{R}^{r \times (2r+d)}$ are jointly parameterized by a single linear projection that is split into gate, candidate, and output components. The state s_0 is initialized to zero.

3.2. Confidence-Scheduled Verification

The semi-autoregressive architecture enables DSpark to generate large draft blocks efficiently. However, producing more draft tokens does not automatically translate to higher end-to-end speedups. Indiscriminately verifying the full draft block can actually degrade overall system throughput, especially in high-concurrency scenarios (Hu et al., 2026b; Liu et al., 2024c).

This performance bottleneck stems from two interacting factors. First, on the data side, draft acceptance rates inherently vary across domains: structured text like code naturally yields high acceptance, whereas open-ended chat has significantly lower acceptance (Abramovich et al., 2026; Xia et al., 2024). Second, on the system side, the actual cost of verifying an extra

token depends strictly on the engine load. Under light system load, an extra verification incurs minimal penalty even if rejected. However, under high-concurrency deployments, every unnecessary verification occupies target model batch capacity that could otherwise serve other active requests (Liu et al., 2024b; Wu et al., 2025).

Therefore, fully unlocking the potential of large draft blocks requires a unified mechanism that routes target model compute only toward tokens with a positive expected return. DSpark achieves this by coupling a **confidence head** (Section 3.2.1) that predicts prefix survival probabilities, with a **hardware-aware prefix scheduler** (Section 3.2.2) that dynamically determines the optimal verification lengths based on current system load.

3.2.1. Confidence Head

Drawing inspiration from Huang et al. (2024); Wang et al. (2026b), the confidence head outputs a scalar $c_k \in (0, 1)$ for each draft position k. Crucially, c_k models the *conditional* probability that the draft token at position k will survive target verification, given that all preceding tokens in the block have been accepted. The architecture features a lightweight linear projection followed by a sigmoid function:

$$c_k = \sigma\big(w^\top [h_k;\, W_1[x_{k-1}]]\big), \tag{7}$$

where h_k is the hidden state of the backbone and $W_1[x_{k-1}]$ is the Markov Embedding from the previous draft token. We supervise c_k using the analytical acceptance rate per-step c_k^*. This rate is determined by the total variation distance between the draft distribution p_k^d and the target distribution p_k^t:

$$c_k^* = 1 - \tfrac{1}{2}\|p_k^d - p_k^t\|_1. \tag{8}$$

Post-hoc Calibration. Unlike threshold-based verification heuristics (Huang et al., 2024; Li et al., 2024b; Zhang et al., 2026b), which only require confidence scores to correctly rank draft token qualities, our hardware-aware scheduling approach (detailed in Section 3.2.2) precisely requires the absolute magnitudes of the cumulative acceptance probabilities to compute the expected acceptance length τ. Because neural confidence estimates are often overconfident (Guo et al., 2017; Ovadia et al., 2019), using the raw confidence scores directly would distort the throughput estimation, leading to suboptimal scheduling.

To address this, we introduce **Sequential Temperature Scaling (STS)**. Because each c_i models a conditional probability, the chain rule dictates that the joint probability of a draft prefix being accepted factorizes into the cumulative product $\prod_{i \leqslant k} c_i$. Using a held-out validation set, STS calibrates this joint probability consecutively from left to right. Specifically, at each position $k \in \{1, \ldots, \gamma\}$, we perform a simple 1D grid search to find the optimal temperature scalar that minimizes the Expected Calibration Error (ECE) (Naeini et al., 2015) of the cumulative product, keeping the already-calibrated scores of all preceding positions fixed. Crucially, temperature scaling is an order-preserving transformation: it rectifies the predicted probabilities to match empirical acceptance rates without disrupting the relative draft token rankings learned by the confidence head.

3.2.2. Hardware-Aware Prefix Scheduler

Prior methods (Huang et al., 2024; Li et al., 2024b) typically apply a static threshold to confidence scores to determine verification length. While effective under isolated, single-request assumptions, static thresholds can be suboptimal in high-concurrency production systems, where the utility of verifying a draft token depends heavily on the current system load.

Algorithm 1 Hardware-Aware Prefix Scheduler

Require: Active requests $r \in \{1, \ldots, R\}$; confidence sequence $c_{r,1}, \ldots, c_{r,\gamma}$ per request; profiled step curve $\mathrm{SPS}(B)$

Ensure: Selected per-request prefix lengths $\ell_1^*, \ldots, \ell_R^*$

for $r = 1$ **to** R **do**
  Compute prefix survival probabilities: $a_{r,j} \leftarrow \prod_{i \leqslant j} c_{r,i}$ for $j = 1, \ldots, \gamma$
end for
Construct candidate space $\mathcal{E} \leftarrow \{(r, j) \mid a_{r,j} > 0\}$ and sort descending by $a_{r,j}$
Initialize states: $\ell_r \leftarrow 0$ for all r; Batch size $B \leftarrow R$; Expected accepts $\tau^* \leftarrow R$
Initialize tracking: $\Theta_{\text{best}} \leftarrow R \cdot \mathrm{SPS}(R)$; Selected lengths $\ell_r^* \leftarrow 0$ for all r
for each $(r, j) \in \mathcal{E}$ in sorted order **do**
  $\ell_r \leftarrow j$; $B \leftarrow B + 1$; $\tau^* \leftarrow \tau^* + a_{r,j}$
  Current throughput $\Theta \leftarrow \tau^* \cdot \mathrm{SPS}(B)$
  if $\Theta > \Theta_{\text{best}}$ **then**
    $\Theta_{\text{best}} \leftarrow \Theta$; Update selected lengths $\ell_r^* \leftarrow \ell_r$
  else
    break
  end if
end for
return $(\ell_1^*, \ldots, \ell_R^*)$ achieving Θ_{best}

To address this, we formulate verification length selection as a global throughput maximization problem (Algorithm 1). Consider a batch of R active requests. For request r, let $c_{r,1}, \ldots, c_{r,\gamma}$ be the per-position confidence estimates, and let $\ell_r \in \{0, \ldots, \gamma\}$ denote the scheduled verification length. Because speculative decoding dynamically accepts draft tokens only as a continuous prefix, the survival probability of a token at position j is the cumulative product $a_{r,j} = \prod_{i \leqslant j} c_{r,i}$.

In a single verification step, the total batch size (measured in tokens) sent to the target model is $B = \sum_{r=1}^{R} (1 + \ell_r)$, and the expected number of successfully accepted tokens is $\tau = \sum_{r=1}^{R} \left(1 + \sum_{j=1}^{\ell_r} a_{r,j}\right)$. Under a simplifying assumption[2], let $\mathrm{SPS}(B)$ denote the engine throughput, measured in steps per second, for a given forward-pass batch size B. Crucially, this capacity curve is profiled once during engine initialization and stored as a lightweight cost table. Our scheduler then aims to maximize the expected system-wide token throughput $\Theta = \tau \cdot \mathrm{SPS}(B)$ by dynamically selecting verification lengths $\ell_1, \ldots, \ell_R$.

Although finding the global maximum of Θ appears to be a combinatorial search, the objective structure allows for an efficient greedy solution. Because $a_{r,j}$ is monotonically non-increasing with respect to j (i.e., $a_{r,j} \leq a_{r,j-1}$), the marginal gain in expected accepted tokens for extending request r's verification length from $j - 1$ to j is exactly $a_{r,j}$. This monotonicity ensures that sorting candidate tokens globally by $a_{r,j}$ naturally respects intra-block prefix dependencies. Consequently, if the total verification batch size B were fixed, the optimal allocation $\{\ell_r\}$ would be determined by greedily selecting the draft tokens with the highest survival probabilities from the global pool of all $\{a_{r,j}\}$.

Building on this insight, the optimization can be evaluated along this greedy admission path.

[2]In practical serving scenarios, average context lengths remain well below extremes (e.g., 1M tokens), making their impact on decode latency marginal for highly optimized architectures like DeepSeek-V4. Moreover, in prefill-decode disaggregated deployments, decode load balancers keep both request counts and total context lengths roughly balanced across data-parallel (DP) ranks. This effectively amortizes sequence-length variance, allowing us to safely assume that engine throughput depends predominantly on the verification batch size B.

We first globally sort all valid prefix extensions in descending order of survival probability. To dynamically determine the optimal target batch size B, we incrementally admit tokens from this sorted pool, updating the expected throughput Θ via an lookup from the cost table.

Lossless speculative decoding strictly requires the *non-anticipating property*: admission decisions must not depend on future candidate tokens (Chen et al., 2023; Leviathan et al., 2023). Because our confidence head relies on the Markov feature of the previously sampled token, computing the next survival probability $a_{r,k+1}$ explicitly requires the instantiated candidate $x_{r,k}$. A retrospective global search would thus inadvertently leak $x_{r,k}$ into the admission decision for step k, introducing selection bias (we provide a concrete counterexample demonstrating this theoretical violation in Appendix A).

To enforce strict causality, the scheduler (Algorithm 1) employs an early-stopping mechanism. By breaking the greedy search immediately when the throughput drops ($\Theta \leq \Theta_{\text{best}}$), the truncation decision relies solely on the prefix processed up to that exact step. This isolates the admission event from future tokens, ensuring exact target-distribution recovery. Note that this stepwise early-stopping yields the global maximum throughput if and only if the objective Θ is unimodal, which implicitly assumes a smoothly decaying hardware capacity curve. We address the engineering adaptations required for real-world, non-smooth SPS characteristics and asynchronous system pipelines in Section 5.2.

3.3. Training

During training, we randomly sample multiple anchor positions from each target sequence to form γ-token blocks as training data. The target model is frozen throughout training; the draft model shares its embedding layer and language modeling head and keeps them frozen, updating only the backbone drafter, sequential block, and confidence head.

The training objective consists of three terms: a cross-entropy loss $\mathcal{L}_{\text{ce}}$, a distribution-matching loss $\mathcal{L}_{\text{tv}}$, and a confidence loss $\mathcal{L}_{\text{conf}}$. All three are position-weighted by $w_k = \exp(-(k-1)/\gamma)$ (Chen et al., 2026), which emphasizes earlier block positions that contribute more to the expected acceptance length under prefix-based verification. The cross-entropy loss $\mathcal{L}_{\text{ce}}$ trains the drafter to predict the correct next token:

$$\mathcal{L}_{\text{ce}} = -\sum_{k=1}^{\gamma} w_k \log p_k^d(x_k^*), \tag{9}$$

where x_k^* is the ground-truth token and p_k^d is the draft distribution. The distribution-matching loss $\mathcal{L}_{\text{tv}}$ penalizes the total variation distance between the draft and target distributions:

$$\mathcal{L}_{\text{tv}} = \sum_{k=1}^{\gamma} w_k \| p_k^d - p_k^t \|_1. \tag{10}$$

Since the total variation distance is a direct proxy for the acceptance rate: the per-step acceptance probability equals $1 - \frac{1}{2}\|p^d - p^t\|_1$ (Leviathan et al., 2023), minimizing $\mathcal{L}_{\text{tv}}$ directly maximizes the expected acceptance rate.

The confidence loss $\mathcal{L}_{\text{conf}}$ is a binary cross-entropy that trains the confidence head to predict the soft acceptance label c_k^* from Equation 8:

$$\mathcal{L}_{\text{conf}} = -\sum_{k=1}^{\gamma} w_k \left[c_k^* \log c_k + (1 - c_k^*) \log(1 - c_k) \right]. \tag{11}$$

The overall objective is a weighted combination of the three terms (with default weights $\alpha_{ce} = 0.1$, $\alpha_{tv} = 0.9$, $\alpha_{conf} = 1.0$):

$$\mathcal{L} = \alpha_{ce}\,\mathcal{L}_{ce} + \alpha_{tv}\,\mathcal{L}_{tv} + \alpha_{conf}\,\mathcal{L}_{conf} \tag{12}$$

4. Experiments

In this section, we validate the draft quality of DSpark using offline benchmarks and report the effectiveness of confidence scheduler under online production traffic in Section 5. The experimental setup is described in Section 4.1, main results in Section 4.2, and additional analyses are included in Section 4.3.

4.1. Experimental Setup

Target and draft models. We evaluate DSpark on four target models spanning different scales and model families: Qwen3-{4B, 8B, 14B} (Yang et al., 2025), and Gemma4-12B (Google DeepMind, 2026). For draft models, we compare DSpark with two representative drafters: DFlash (Chen et al., 2026), a state-of-the-art parallel drafter, and Eagle3 (Li et al., 2026b), an autoregressive drafter based on Training-Time Test (TTT). For strict and fair comparison, we retrain all drafters in the same training framework and on the same data[3]. We align Eagle3's TTT horizon (7) with the block size (7) used by DFlash and DSpark, and we use the same target-model feature layers for all drafters. For the number of draft model layers, we set 1 for Eagle3 and 5 for DSpark and DFlash (Chen et al., 2026). Unless otherwise stated, DSpark denotes the Markov-head variant; we study the RNN-head variant in Section 4.3.2.

Training data. We use Open-PerfectBlend, an open-sourced version of PerfectBlend (Xu et al., 2024) consisting of 1.3 million samples. It is a general-purpose instruction dataset containing chat (17.6%), math (39.4%), code (38.9%), and instruction-following data (4.1%). We only use the prompts from Open-PerfectBlend; responses are regenerated by each target model with recommended sampling parameters. Each drafter is trained for 10 epochs to ensure full convergence. For data generation and evaluation, we adopt the non-thinking mode.

Evaluation protocol. We evaluate the performance of different algorithms on three domains:

1. **Mathematical Reasoning**, including GSM8K (Cobbe et al., 2021), MATH500 (Lightman et al., 2024) and AIME25 (Zhang and Math-AI, 2025).
2. **Code Generation**, including MBPP (Austin et al., 2021b), HumanEval (Chen et al., 2021) and Live-CodeBench (Jain et al., 2025).
3. **Daily Chat**, including MT-Bench (Zheng et al., 2023), Alpaca (Taori et al., 2023) and Arena-Hard (Li et al., 2024a, 2025b).

For all benchmarks, we use standard speculative decoding (Chen et al., 2023; Leviathan et al., 2023) with the sampling temperature set to 1.0. We report the accepted length (τ) per decoding round[4]. For all drafters, we use chain-based drafting.

[3] To facilitate future research, we release all checkpoints we trained, including Eagle3, DFlash and DSpark.

[4] For clarity, unless otherwise stated, all reported metrics for accepted length and acceptance rate include the target-generated bonus token.

Table 1 | **Main speculative decoding results.** We report accepted length (τ) per decoding round (higher is better) for different target models and domains. **Bold** marks the best results.

Target	Drafter	Math			Code			Chat		
		GSM8K	MATH	AIME25	MBPP	HumanEval	LCB	MT-Bench	Alpaca	Arena-Hard
Qwen3-4B	Eagle3	5.14	4.62	3.92	3.69	4.16	3.77	2.39	2.26	2.55
	DFlash	5.40	4.85	4.15	4.40	4.74	4.18	3.07	2.96	2.83
	DSpark	**6.11**	**5.70**	**4.89**	**5.13**	**5.38**	**4.86**	**3.64**	**3.54**	**3.29**
Qwen3-8B	Eagle3	5.30	4.77	3.91	3.96	4.33	4.17	2.66	2.54	2.54
	DFlash	5.33	4.91	4.07	4.36	4.64	4.39	3.11	2.98	2.81
	DSpark	**6.17**	**5.78**	**5.01**	**5.16**	**5.52**	**5.17**	**3.72**	**3.58**	**3.21**
Qwen3-14B	Eagle3	5.24	4.60	3.71	3.81	4.14	4.01	2.62	2.47	2.48
	DFlash	5.41	4.84	3.98	4.44	4.59	4.33	3.10	2.94	2.72
	DSpark	**6.21**	**5.74**	**4.94**	**5.26**	**5.43**	**5.02**	**3.70**	**3.58**	**3.13**
Gemma4-12B	Eagle3	5.87	5.46	4.83	4.72	5.37	4.16	3.19	3.06	2.72
	DFlash	5.45	5.04	4.22	4.39	4.95	3.70	2.98	2.84	2.59
	DSpark	**6.05**	**5.78**	**5.12**	**5.11**	**5.64**	**4.51**	**3.49**	**3.35**	**2.92**

4.2. Experimental Results

To isolate the raw draft quality from system-level scheduling policies, our offline evaluation disables the confidence scheduler, forcing all drafters to propose a fixed block of tokens. The main results, measured by the average accepted length (τ) per round, are reported in Table 1.

DSpark consistently outperforms both the autoregressive baseline (Eagle3) and the parallel baseline (DFlash) across all evaluated target models and benchmark domains. Specifically, across the Qwen3-4B, 8B, and 14B models, DSpark improves the macro-average accepted length over Eagle3 by 30.9%, 26.7%, and 30.0%, respectively. Similarly, compared to DFlash, DSpark yields relative improvements of 16.3%, 18.4%, and 18.3% across the three scales. Crucially, this advantage generalizes across model families, as demonstrated by the consistent performance gains on the Gemma4-12B target.

Beyond the average improvements, Table 1 reveals a strong domain effect: the accepted length is naturally higher on structured tasks (e.g., 5.57 on math and 5.12 on code for Qwen3-4B) than on open-ended chat (3.49). This inherent variance in data predictability means a static verification length often wastes compute on trailing tokens that are highly likely to be rejected. This directly motivates our confidence-scheduled verification, which dynamically prunes the draft block based on expected acceptance.

4.3. Experimental Analysis

4.3.1. Why Can Parallel Generation Outperform Autoregression?

Table 1 presents a counter-intuitive observation: the parallel drafter (DFlash) and the semi-autoregressive drafter (DSpark) often yield longer accepted lengths than the fully autoregressive drafter (Eagle3). This finding contrasts with the standard expectation that step-by-step autoregression produces higher-quality sequences than parallel models (Israel et al., 2026; Ren et al., 2020; Zheng et al., 2025).

To analyze this behavior, we examine performance beyond the macro-level accepted length. Using the Qwen3-4B target model and the benchmark sets described in Section 4.1, we introduce *position-wise conditional acceptance* tracked during actual speculative decoding rollouts. Specifically, for a given draft position k, the evaluation denominator counts only the instances where

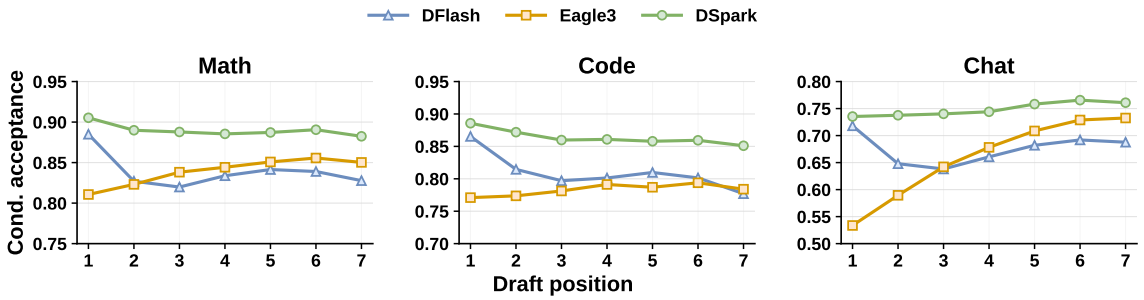

Figure 2 | **Position-wise conditional acceptance.** We report the empirical conditional acceptance rate for each draft position, averaged across benchmarks within each domain using the Qwen3-4B target model. Unlike standard prefix survival, this metric isolates the baseline predictive quality at position k by removing the penalty of previous rejections. Notice that the autoregressive drafter (Eagle3) remains stable or trends upward, while the parallel drafter (DFlash) suffers suffix decay.

the target model successfully verifies and accepts all preceding draft tokens from 1 to $k-1$. The metric then calculates the proportion of these valid instances where the token at position k is also accepted. This approach ensures that the evaluation of position k is not penalized by earlier prefix errors, revealing the underlying predictive quality at each specific step. Figure 2 details these measurements, demonstrating clear behavioral differences across the architectures.

The Capacity Advantage at Position 1. At the first draft position, both architectures predict the next token based solely on the target context. The performance divergence here stems strictly from architectural capacity: autoregressive models like Eagle3 are constrained to shallow networks due to their $O(\gamma)$ latency, whereas $O(1)$ parallel drafters can afford much deeper networks. This structural gap yields a substantial accuracy margin at position 1, with DFlash starting noticeably higher than Eagle3 (e.g., 0.88 vs. 0.81 on Math, and 0.72 vs. 0.53 on Chat). Because speculative decoding operates as a strict prefix-matching survival process, the first token carries the highest leverage—a rejection here immediately invalidates the entire block. Consequently, this initial capacity advantage disproportionately boosts the final accepted length, explaining why parallel drafters ultimately outperform autoregressive ones globally despite rapid acceptance decay at later positions.

The Limitation of Independence at Later Positions. Examining the tail of the curves (positions 2 through 7) exposes the inherent limitation of independent parallel generation. As earlier tokens lock in a specific semantic path, subsequent tokens naturally become more predictable. Autoregressive models like Eagle3 effectively leverage this conditional certainty, maintaining or even increasing conditional acceptance deeper into the block (e.g., from 0.53 to 0.74 on Chat). In contrast, DFlash suffers from rapid acceptance decay, dropping from 0.87 to 0.78 on Code and 0.72 to 0.63 on Chat. Because each parallel position marginalizes over all possible prior tokens rather than conditioning on an exact sampled prefix, the model frequently proposes inconsistent suffix combinations—a mode known as multi-modal collision (Gu et al., 2018; Stern et al., 2018).

Mitigating Suffix Decay with Semi-Autoregression. The preceding analysis highlights a clear architectural objective: combining the high capacity of a parallel backbone for the initial token with the dependency modeling of an autoregressive model for subsequent tokens. This directly

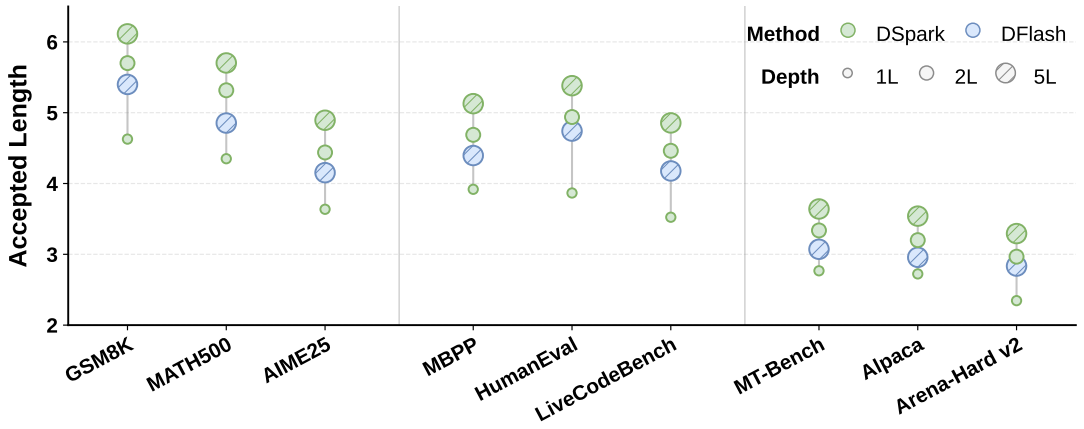

Figure 3 | **Effect of drafter depth.** With proposal length fixed, DSpark's performance improves as drafter layers are added. Notably, a shallow 2-layer DSpark outperforms a deeper 5-layer DFlash baseline, highlighting the parameter efficiency of sequential modeling.

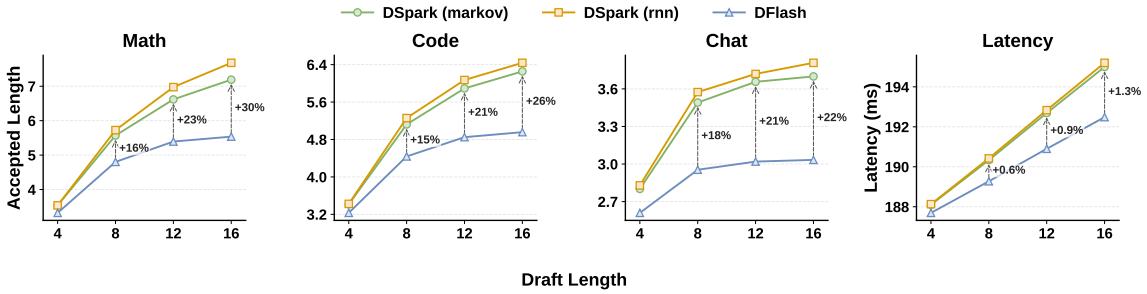

Figure 4 | **Effect of proposal length and latency overhead.** DSpark consistently outperforms DFlash across various block sizes (left three panels). The rightmost panel demonstrates that the sequential head introduces minimal latency overhead during serving.

motivates DSpark's semi-autoregressive design. As shown in Figure 2, DSpark inherits the high initial acceptance of the deep parallel drafter (e.g., starting at 0.93 on Math). Simultaneously, its lightweight sequential head mitigates the rapid acceptance decay typical of parallel generation. By resolving this trade-off, DSpark maintains a high and stable conditional acceptance rate throughout the entire draft block.

4.3.2. A Little Autoregression Goes a Long Way

Building on the insights from Section 4.3.1, we explore the architectural design space of DSpark along two dimensions: drafter depth (number of transformer layers) and proposal length (block size γ). Unless otherwise stated, all experiments in this section use Qwen3-4B as the target model and follow the evaluation protocol detailed in Section 4.1.

Drafter Depth. Increasing the number of transformer layers naturally expands a draft model's predictive capacity. To isolate this effect, we fix the block size to 7 and vary the number of DSpark layers from 1 to 5, comparing it against a 5-layer DFlash baseline. Figure 3 aggregates the accepted lengths across the math, code, and chat domains. As expected, DSpark's performance improves monotonically with depth, with the steepest marginal gain occurring from one to two layers. Notably, a 2-layer DSpark outperforms the 5-layer DFlash baseline across all domains.

This demonstrates that injecting local auto-regression via a lightweight sequential head offers a highly favorable accuracy-parameter trade-off, achieving better sequence coherence than simply stacking deeper parallel layers.

Proposal Length. Next, we fix the drafter depth to 5 layers and scale the draft length (proposal length γ plus one anchor token) across $\{4, 8, 12, 16\}$ to evaluate performance on longer draft blocks. For DSpark, we evaluate both the default Markov head and the RNN head. The first three panels of Figure 4 show that DSpark consistently outperforms DFlash at every proposal length. More importantly, the performance gap steadily widens as γ increases. Because pure parallel generation (DFlash) suffers from rapid acceptance decay (Figure 2), its marginal utility diminishes for long blocks. DSpark mitigates this decay, causing its relative gain over DFlash to grow. For instance, at $\gamma = 7$, DSpark improves the accepted length by 16% on math, 15% on code, and 18% on chat; at $\gamma = 15$, these gains expand to 30%, 26%, and 22%, respectively. Also, RNN head provides only marginal additional gains over the Markov head, mainly at longer proposal lengths. Given its higher implementation complexity and less favorable deployment properties, we use the Markov head as the default.

Latency Overhead. We quantify the overhead of the sequential generation loop in DSpark. The rightmost panel of Figure 4 reports the per-round engine latency—comprising one target verification pass, the parallel draft block forward, and the serial sampling loop—measured at a batch size of 128. To prevent sequence-length bias, the reported latency represents the arithmetic mean across varying context lengths ($\{512, 1024, 2048, 4096\}$ tokens). Since the target model dominates the verification compute time at this batch size, the sequential block's latency overhead is negligible. Consequently, scaling the draft length from 4 to 16 adds a marginal 0.2% to 1.3% to the full-round latency over the DFlash baseline, despite delivering up to a 30% improvement in accepted length.

4.3.3. Verify Smarter, Not Longer: The Role of Confidence Head

While DSpark sustains high acceptance over long draft blocks, verifying the entire proposal remains inefficient (Hu et al., 2026b; Huang et al., 2024). Due to the inherent domain variance noted in Section 4.2, trailing tokens in open-ended chat still face high rejection risks, making blind verification a waste of target compute. To evaluate whether the confidence head can effectively prune these unpromising suffixes, we conduct an offline threshold sweep using Qwen3-4B. We validate the estimator in isolation here, reserving the hardware-aware prefix scheduler (Section 3.2.2) for live production evaluation in Section 5.

Diagnostic: Static Threshold Sweep. Figure 5 plots the average tokens per step (bars) and the overall acceptance rate (line) across confidence thresholds. As the threshold increases, the acceptance rate steadily rises because the estimator filters out tokens that would ultimately be rejected (hashed bars). This suggests that the confidence head can identify lower-value suffix tokens and this pruning is most pronounced on chat workloads, where higher-entropy token distributions limit the efficiency of fixed-length verification. In the Chat subplot, raising the threshold significantly reduces rejected tokens, increasing the acceptance rate from 45.7% to 95.7%. In contrast, structured tasks (Math and Code) experience milder pruning and retain more draft tokens, with acceptance rates rising from 76.9% to 92.5% and 67.6% to 92.0%, respectively.

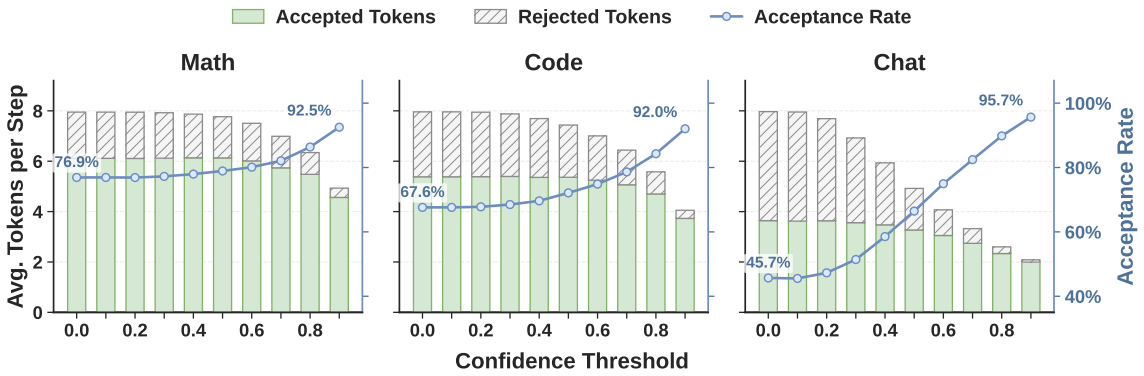

Figure 5 | **Confidence threshold sweep.** A threshold of 0 corresponds to standard fixed-length verification. As the threshold increases, the overall acceptance rate steadily rises because the confidence head effectively prunes tokens that would ultimately be rejected (hashed bars).

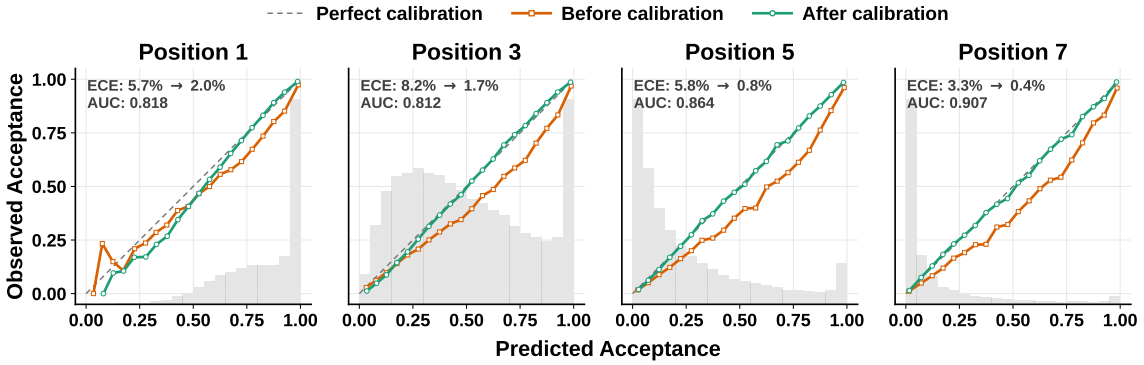

Figure 6 | **The Reliability Diagram on Alpaca Dataset.** While the raw confidence estimator achieves strong discrimination, its predictions are inherently overconfident. Applying post-hoc calibration helps to align the prefix survival probabilities with empirical acceptance rates. The shaded background histogram represents the frequency distribution of sample counts across different confidence bins.

From Static Thresholds to Calibrated Scheduling. While useful for diagnostics, a static threshold is sub-optimal in dynamic serving environments because it ignores system load: verifying low-confidence tokens incurs minimal opportunity cost under low concurrency, but wastes critical batch capacity under high concurrency. This load dependency motivates the hardware-aware prefix scheduler. As formulated in Section 3.2, maximizing system-level throughput requires the confidence model to exhibit both strong predictive discrimination and precise calibration to accurately estimate cumulative survival probabilities. The reliability diagram (Figure 6) demonstrates that while the raw model achieves strong discrimination (ROC-AUC (Hanley and McNeil, 1982) ranging from 0.81 to 0.90), it is overly confident (ECE 3%–8%). Applying post-hoc STS (Section 3.2.1) mitigates this overconfidence, reducing the average ECE to ~1% and yielding reliable survival estimates.

5. Real-World Deployment of DSpark

While Section 4 establishes the algorithmic gains of DSpark on offline benchmarks, deploying it alongside large-scale models like DeepSeek-V4 (DeepSeek-AI, 2026) introduces additional

system-level challenges across both training and inference. In this section, we present the end-to-end production pipeline of DSpark. We detail our scalable training mechanisms, the system-level optimizations necessary to deploy the hardware-aware prefix scheduler (Section 3.2.2), and the framework's end-to-end performance under live user traffic.

5.1. Scalable and Flexible Training

The DSpark draft models are co-deployed with the preview versions of DeepSeek-V4-Flash and DeepSeek-V4-Pro (DeepSeek-AI, 2026). The parallel backbone comprises three MoE layers (Dai et al., 2024) with mHC (Xie et al., 2026) and a sliding window attention of 128. We configure the maximum block size to $\gamma = 5$ and utilize the Markov head for sequential modeling. Furthermore, the confidence head is trained end-to-end alongside the draft model and subsequently calibrated via STS to provide reliable scheduling signals.

Training the draft model requires the target model's output distributions for supervision. Evaluating both models over the full document context incurs substantial memory footprints and inter-worker communication overhead. To address these bottlenecks, we implement two system-level optimizations within our internal training framework (HAI-LLM)[5]:

- **Hidden state communication.** Transferring the target model's full-vocabulary logits ($V \approx 10^5$) across parallel workers creates a significant bandwidth bottleneck. Instead, we temporarily cache the target model's forward-pass activations and communicate only the hidden states immediately preceding the language modeling (LM) head. The LM head projection is then executed locally on the draft model's workers only for the sampled target positions. This reduces the per-token communication complexity to $O(d)$, where d is the hidden dimension.
- **Anchor-bounded sequence packing.** To decouple the draft model's computational cost from the target model's context length, we sample a fixed number of draft anchors from the training sequence and pack these isolated prediction blocks into dense training batches. We manage this packing via token-level attention indices rather than standard 2D masks. This maintains exact causal masking across multiple independent sequences and anchors, avoiding the computational and memory overhead associated with standard padding.

5.2. Hardware-Aware Prefix Scheduler in Practice

In Section 3.2.2, Algorithm 1 provides a theoretically sound and lossless scheduling mechanism. However, directly deploying this algorithm into a production environment exposes two fundamental conflicts with real-world infrastructure. First, the algorithm assumes a smooth, unimodal capacity curve, whereas the true hardware capacity $\mathrm{SPS}(B)$ is inherently discrete, exhibiting a jagged, step-wise degradation (Yan et al., 2020). Second, the algorithm requires scheduling of dynamic draft tokens per step, which clashes with continuous CUDA graph replay (Fireworks AI, 2023) and Zero-Overhead Scheduling (ZOS) (Zheng et al., 2024; Zhu et al., 2025).

To navigate the trade-offs among system compatibility, throughput, and algorithmic correctness, we adapt the scheduler to operate asynchronously. Because ZOS requires the batch size for the next step to be known before the current step completes, synchronous scheduling would inevitably stall the GPU pipeline. Instead, we approximate the upcoming verification capacity using the confidence head outputs from two steps prior. Mechanically, the candidate tokens in the current step are still strictly sorted by their actual, up-to-date cumulative confidence scores; the historical prediction from two steps prior is used solely to determine the dynamic truncation

[5] https://www.high-flyer.cn/en/blog/hai-llm/

length (i.e., the batch capacity limit K). This effectively casts the admission process as a dynamic top-K selection. While approximating the capacity K introduces a slight temporal offset, the selection mechanism is fundamentally rank-preserving: the most confident draft tokens are always prioritized for verification. This adaptation fully hides scheduling latency and ensures seamless ZOS integration.

Building on this asynchronous pipeline, we resolve the hardware utilization bottleneck. To prevent the scheduler from being trapped in local minima by jagged SPS cliffs, we remove the early-stopping `break`, enabling an unconstrained global search. Ordinarily, this retrospective search would leak future token information and violate the lossless guarantee (Appendix A). However, our ZOS-driven adaptation naturally prevents this. Because the unconstrained search evaluates only historical predictions from two steps prior, the admission decision is isolated from the realization of the current token $x_{r,k}$. The truncation length inherently depends only on information available from two steps prior. Thus, asynchronous design forms a causal barrier, maximizing physical throughput across hardware cliffs while preserving the exact target distribution.

5.3. High-Throughput and Low-Latency Inference

During decoding, production serving systems must simultaneously optimize two competing objectives: per-request latency and aggregate throughput (Kwon et al., 2023; Zhao et al., 2025a; Zhong et al., 2024). The former governs the quality of service for individual users—a factor increasingly critical in agent-based workloads (Tiwari et al., 2026)—while the latter determines the total number of concurrently served users. Because speculative decoding inevitably incurs wasted verification compute, it inherently navigates this trade-off, trading extra system compute for faster per-request generation.

In our deployment setting, however, the number of requests processed per step is frequently constrained by resource limits (e.g., fixed KV-cache capacity per request) and the pool of available user traffic (e.g., RL long-tail loads). Consequently, the effective batch size persistently remains well below the GPU's compute-saturating threshold. Under this regime, the traditional trade-off simplifies: given a fixed concurrency limit, maximizing per-GPU total token throughput and maximizing the generation speed per user (*tok/s/user*) become highly correlated objectives rather than competing ones.

To achieve this maximum throughput, the asynchronous scheduler (Section 5.2) actively routes idle compute toward the most promising draft tokens. However, executing this dynamic routing introduces a severe challenge at the physical execution layer: the inference framework must efficiently support variable-length queries within a single batch. Standard decode kernels are heavily optimized for fixed query lengths; naively processing variable-length verified prefixes leads to severe GPU under-utilization due to padding and uneven workload distribution. We resolve this by decoupling physical execution from logical sequence tracking. In our compute kernels, all tokens across different requests are flattened and processed identically as independent elements. The complex intra-sequence dependencies are then strictly conveyed via a marker tensor integrated into our sparse attention implementation. Specifically on the DeepSeek-V4 architecture, only the index-attention and compress kernels require modification to support this variable-length routing, allowing the dynamic scheduler to operate seamlessly without introducing low-level execution overhead.

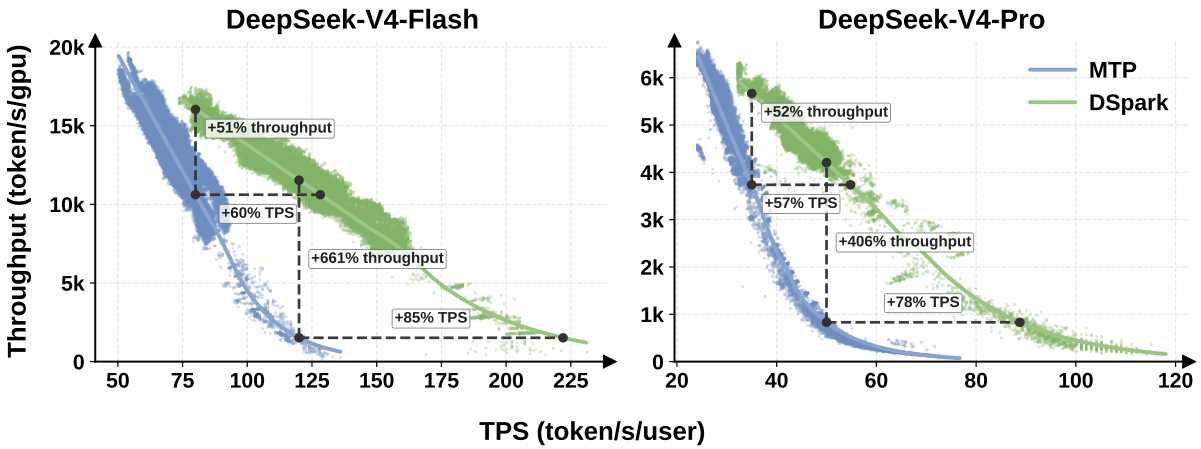

Figure 7 | **Throughput vs. TPS.** Aggregate output token throughput against per-request generation speed (tok/s/user) under live traffic. In our production deployment, DSpark improves the observed throughput–interactivity frontier relative to the MTP-1 baseline under the measured traffic and engine configurations.

5.4. Performance under Live User Traffic

We evaluate DSpark-5 (configured with a maximum draft length of $\gamma = 5$) against the MTP-1 (DeepSeek-AI, 2024) baseline within the production serving engines of DeepSeek-V4-Flash (preview) and DeepSeek-V4-Pro (preview). MTP-1 represents the former production setup, having been superseded by DSpark two weeks following the DeepSeek-V4-preview release. This single-token setup was historically maintained in production because deploying a static multi-token drafter (e.g., MTP-3/5) strictly degrades aggregate throughput under high concurrency due to excessive verification overhead. Therefore, comparing DSpark against this established baseline directly demonstrates its ability to safely unlock the performance potential of larger draft blocks in dynamic serving environments. In all figures, the scatter points represent raw telemetry data sampled directly from live user traffic, capturing complex, real-world request distributions, while the solid lines represent the fitted performance frontiers.

The Serving Pareto Frontier. Figure 7 illustrates the trade-off between aggregate system throughput and per-user generation speed (interactivity). To quantify DSpark's behavior under practical deployment constraints, we evaluate the system at several interactivity SLA anchors. Here, an SLA (Service Level Agreement) specifies the minimum per-user generation speed (in tokens per second) that the system must guarantee.

For the V4-Flash engine, we evaluate the system at SLA anchors of 80 and 120 tok/s/user. At the moderate 80 tok/s/user SLA, DSpark improves aggregate throughput by 51% over the MTP-1 baseline. The stricter 120 tok/s/user SLA represents a qualitatively different regime: under this constraint, the single-token MTP-1 baseline approaches its operational boundary and can sustain only a very small concurrent batch. Consequently, the relative throughput ratio at this point is numerically large, with DSpark achieving a nominal 661% higher aggregate throughput. We therefore interpret this high-SLA point primarily as evidence that DSpark extends the feasible interactivity frontier, rather than as a representative multiplicative speedup over a well-utilized baseline. At matched practical throughput levels, which provide a more stable comparison, DSpark accelerates per-user generation speeds by 60% to 85%.

The V4-Pro deployment shows the same pattern. At the moderate 35 tok/s/user SLA, DSpark improves aggregate throughput by 52%. At the stricter 50 tok/s/user SLA, MTP-1 again enters a low-concurrency regime, yielding a nominal 406% relative throughput advantage for DSpark. As with V4-Flash, we treat this point as an indication that DSpark sustains useful throughput under an interactivity target that the baseline cannot efficiently support. At matched system capacities, DSpark delivers 57% to 78% faster per-user generation. Overall, these results show that DSpark shifts the observed throughput–interactivity frontier outward: it improves throughput in moderate-SLA regimes and, more importantly, preserves non-degenerate serving capacity under strict interactivity constraints.

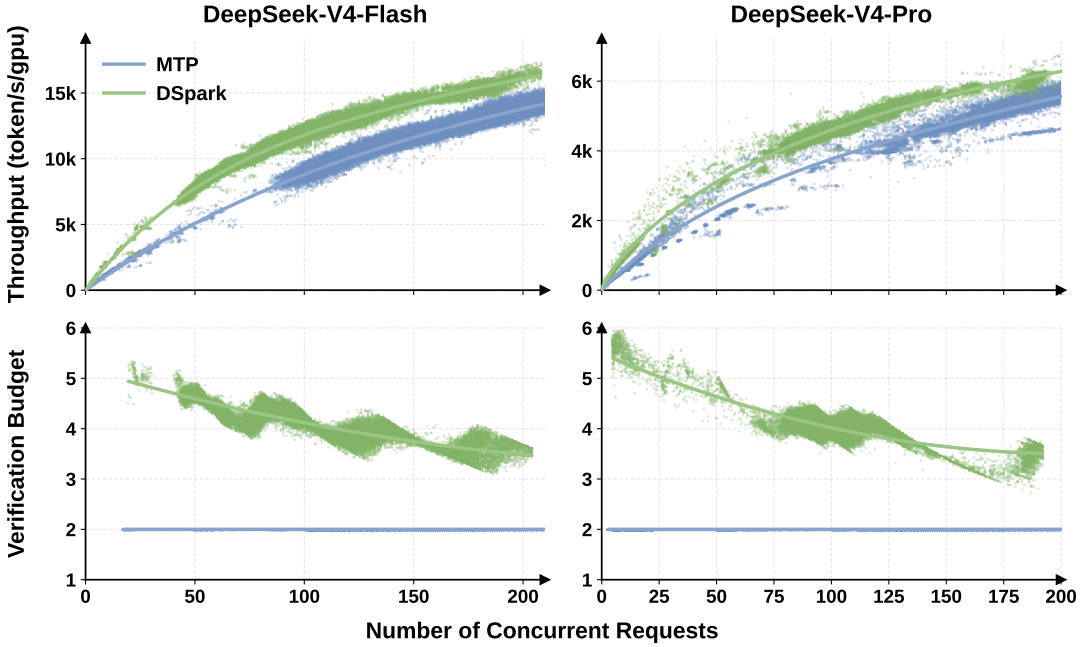

Figure 8 | **Load-adaptive throughput and verification budgets.** Top row (a, b): Aggregate output throughput across varying levels of system concurrency. Bottom row (c, d): The average target verification budget allocated per request. As concurrent load increases, the dynamic scheduler automatically restricts the per-request verification length to prevent resource contention.

Throughput Dynamics under Load. Figure 8 analyzes the underlying mechanism driving these gains by plotting aggregate throughput (top row) and the dynamic verification budget (bottom row) against system concurrency.

- Under the moderate concurrency regimes typical of our production deployment (fewer than 200 concurrent requests for V4-Flash and 150 for V4-Pro), the hardware-aware scheduler leverages available target compute capacity by allocating longer verification budgets, expanding from MTP-1's static 2 tokens to roughly 4–6 tokens per request. This extended verification yields more accepted tokens per forward pass, directly contributing to the throughput gains observed on the Pareto frontier.
- As system concurrency scales and target capacity saturates, the scheduler dynamically restricts this budget. The average verification length decreases smoothly with load, ensuring that low-confidence draft tokens are pruned before they consume critical batch capacity. This load-aware behavior stabilizes production deployment: DSpark maximizes

the utility of idle compute under light traffic, while effectively preserving critical batch capacity under heavy traffic.

Limitations. Although the prefix scheduler minimizes wasted target-model verification, DSpark still incurs a fixed draft-side cost to generate the initial γ-token block via the parallel backbone. For complex queries with inherently low acceptance rates, this upfront drafting compute is unrecoverable. Future optimizations could introduce difficulty-aware early exiting within the draft model, enabling such requests to bypass full-block generation.

6. Related Work

Speculative Decoding Algorithms. Speculative decoding accelerates autoregressive generation by decoupling token proposal from verification. Building on early blockwise methods (Ge et al., 2022; Stern et al., 2018; Sun et al., 2021; Xia et al., 2023), modern approaches employ rejection sampling to exactly preserve the target model's distribution (Chen et al., 2023; Leviathan et al., 2023). Because inference speedup directly depends on the drafter's efficiency and accuracy, extensive research has focused on optimizing its architecture. Beyond using standalone small language models (Chen et al., 2023; Leviathan et al., 2023), subsequent work integrates multi-token heads or feature extrapolators directly into the target model (Ankner et al., 2024; Cai et al., 2024, 2025; DeepSeek-AI, 2024; Eldenk et al., 2026; Gloeckle et al., 2024; Li et al., 2024b,c, 2026b; Zhang et al., 2025). Other strategies include self-speculation via early exits (Elhoushi et al., 2024; Liu et al., 2024a; Xia et al., 2025; Zhang et al., 2024), dynamic vocabulary compression (Williams et al., 2026; Zhao et al., 2025b), prompt lookup (Saxena, 2023; Somasundaram et al., 2025), suffix automata (Hu et al., 2025), and retrieval (He et al., 2023; Shen et al., 2026). To remove the sequential bottleneck of the drafting itself, a line of research proposes parallel or blockwise generation, including Medusa (Cai et al., 2024), P-EAGLE (Hui et al., 2026), PARD (An et al., 2026a,b), DART (Liu et al., 2026a) and DFlash (Chen et al., 2026). DDTree, TAPS and JetSpec then extend the draft chain to verifiable trees (Hu et al., 2026a; Ringel and Romano, 2026; Wang et al., 2026a). Concurrent efforts include: Domino (Huang et al., 2026a) introduces a CausalEncoder conceptually similar to our RNN Head; DFlare (Zhang et al., 2026a) addresses conditioning bottlenecks via layer-wise fusion.

System-Aware Scheduling for Speculative Decoding. Beyond drafter architecture, another line of work focuses on determining the optimal number of speculative tokens to generate or verify in each round. To this end, various approaches adapt draft lengths on the fly using confidence heuristics (Du et al., 2024; Li et al., 2024b; Liu et al., 2026c; Mamou et al., 2024; Wen and Feng, 2026), learned acceptance predictors (Huang et al., 2024; Zacks917, 2026), or bandit-style policies (Liu et al., 2026b). Furthermore, recognizing speculative decoding as inherently a system-level scheduling problem, recent works optimize overall goodput and latency by adjusting speculation budgets according to real-time system load and request priority (AngelSlim Team, 2026; Hu et al., 2026b; Huang et al., 2026b; Li et al., 2026a; Liu et al., 2024c; Miao et al., 2024; Sadhukhan et al., 2025; Wu et al., 2025).

Parallel Generation. Models that generate tokens in parallel offer a decoding latency nearly independent of output length, making them an attractive alternative to autoregressive decoding. Non-Autoregressive Transformers (NATs, Gu et al., 2018) pioneered this direction by predicting all positions independently in a single pass. However, this forces the model to average over all

plausible modes, often producing outputs that mix fragments from different valid sequences. Two broad lines of work have emerged to address this limitation. One direction retains the single-pass architecture but changes what the model sees or how it is trained: introducing latent variables as conditioning input to steer all positions toward a consistent output (Gu et al., 2018; Kaiser et al., 2018; Ma et al., 2019), or relaxing the training objective so that the model focuses on producing a single coherent output rather than modeling the full distribution over all valid alternatives (Du et al., 2021; Qian et al., 2021; Shao et al., 2021, 2023). The other direction reintroduces limited sequential dependency through iterative re-prediction (Austin et al., 2021a; Ghazvininejad et al., 2019; Li et al., 2022), block-level autoregression (Arriola et al., 2025; Wang et al., 2018), or structured output layers such as CRF (Sun et al., 2019), CTC (Libovický and Helcl, 2018; Saharia et al., 2020), HMM (Huang et al., 2022b), and PCFG (Gui et al., 2023).

Speculative decoding places a further demand that the drafter must provide exact per-token probabilities for the rejection sampling rule. Most techniques above cannot readily provide such probabilities due to iterative refinement, latent marginalization, or global normalization. For instance, in a design closely related to ours, CRF-NAT (Sun et al., 2019) also places a sequential module over parallel hidden states, but its globally normalized partition function prevents exact per-token probability computation. Similarly, when adapting the CTC output layer to parallel speculative decoding, CTC-drafter (Wen et al., 2024) is restricted to greedy verification due to the latent marginalization of alignment paths. DSpark circumvents these limitations by keeping the sequential correction local, so per-token probabilities remain exact softmax evaluations.

7. Conclusion

In this paper, we present DSpark, a speculative decoding framework designed to overcome the structural and system-level bottlenecks of large language model inference in high-concurrency production environments. Algorithmically, DSpark introduces a semi-autoregressive generation paradigm—coupling a computationally heavy parallel backbone with a lightweight sequential head—to mitigate the rapid suffix decay of independent parallel drafters. At the system level, we formulate verification length selection as a global throughput maximization problem, employing a hardware-aware prefix scheduler that dynamically tailors the target model's verification budget based on calibrated survival probabilities and real-time engine load. Extensive offline evaluations demonstrate that DSpark substantially outperforms state-of-the-art autoregressive and parallel baselines across diverse domains. Furthermore, its real-world deployment within the DeepSeek-V4 validates its practical value in production serving: by intelligently managing verification overhead, DSpark sustains robust concurrency under heavy load, consistently accelerates per-user generation speeds, and shifts the Pareto frontier of LLM serving outward.

References

T. Abramovich, M. Ashkenazi, I. Putterman, B. Chislett, T. Mitra, B. D. Rouhani, R. Zilberstein, and Y. Geifman. Speed-bench: A unified and diverse benchmark for speculative decoding. arXiv preprint arXiv:2604.09557, 2026.

Z. An, H. Bai, Z. Liu, D. Li, and E. Barsoum. PARD: Accelerating LLM inference with low-cost PARallel draft model adaptation. In The Fourteenth International Conference on Learning Representations, 2026a. URL https://openreview.net/forum?id=XbOyv7iVGL.

Z. An, T. Liu, Z. Liu, D. Li, R. Liu, and E. Barsoum. Pard-2: Target-aligned parallel draft model for dual-mode speculative decoding. arXiv preprint arXiv:2605.08632, 2026b.

AngelSlim Team. D-Cut: Adaptive verification depth pruning for speculative decoding, 2026. URL https://angelslim.readthedocs.io/zh-cn/latest/dcut.html.

Z. Ankner, R. Parthasarathy, A. Nrusimha, C. Rinard, J. Ragan-Kelley, and W. Brandon. Hydra: Sequentially-dependent draft heads for medusa decoding. In First Conference on Language Modeling, 2024. URL https://openreview.net/forum?id=FbhjirzvJG.

M. Arriola, S. S. Sahoo, A. Gokaslan, Z. Yang, Z. Qi, J. Han, J. T. Chiu, and V. Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. In The Thirteenth International Conference on Learning Representations, 2025. URL https://openreview.net/forum?id=tyEyYT267x.

J. Austin, D. D. Johnson, J. Ho, D. Tarlow, and R. van den Berg. Structured denoising diffusion models in discrete state-spaces. In A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, editors, Advances in Neural Information Processing Systems, 2021a. URL https://openreview.net/forum?id=h7-XixPCAL.

J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, et al. Program synthesis with large language models. arXiv preprint arXiv:2108.07732, 2021b.

T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, editors, Proceedings of the 41st International Conference on Machine Learning, volume 235 of Proceedings of Machine Learning Research, pages 5209–5235. PMLR, 21–27 Jul 2024. URL https://proceedings.mlr.press/v235/cai24b.html.

Y. Cai, X. Liang, X. Wang, J. Ma, H. Liang, J. Luo, X. Zuo, L. Duan, Y. Yin, and X. Chen. Fastmtp: Accelerating llm inference with enhanced multi-token prediction, 2025. URL https://arxiv.org/abs/2509.18362.

C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper. Accelerating large language model decoding with speculative sampling. arXiv preprint arXiv:2302.01318, 2023.

J. Chen, Y. Liang, and Z. Liu. Dflash: Block diffusion for flash speculative decoding. arXiv preprint arXiv:2602.06036, 2026.

M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code, 2021.

Y. Cheng, A. Zhang, X. Zhang, C. Wang, and Y. Wang. Recurrent drafter for fast speculative decoding in large language models, 2024. URL https://arxiv.org/abs/2403.09919.

K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.

D. Dai, C. Deng, C. Zhao, R. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 1280–1297, 2024.

DeepSeek-AI. Deepseek-v3 technical report. arXiv preprint arXiv:2412.19437, 2024.

DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026. URL https://arxiv.org/abs/2606.19348.

C. Du, Z. Tu, and J. Jiang. Order-agnostic cross entropy for non-autoregressive machine translation. In M. Meila and T. Zhang, editors, Proceedings of the 38th International Conference on Machine Learning, volume 139 of Proceedings of Machine Learning Research, pages 2849–2859. PMLR, 18–24 Jul 2021. URL https://proceedings.mlr.press/v139/du21c.html.

C. Du, J. Jiang, X. Yuanchen, J. Wu, S. Yu, Y. Li, S. Li, K. Xu, L. Nie, Z. Tu, and Y. You. GliDe with a CaPE: A low-hassle method to accelerate speculative decoding. In R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, editors, Proceedings of the 41st International Conference on Machine Learning, volume 235 of Proceedings of Machine Learning Research, pages 11704–11720. PMLR, 21–27 Jul 2024. URL https://proceedings.mlr.press/v235/du24c.html.

D. Eldenk, P. Mohapatra, Y. Comlek, K. Oktay, H. Zhang, and S. Xia. Attention drift: What autoregressive speculative decoding models learn, 2026. URL https://arxiv.org/abs/2605.09992.

M. Elhoushi, A. Shrivastava, D. Liskovich, B. Hosmer, B. Wasti, L. Lai, A. Mahmoud, B. Acun, S. Agarwal, A. Roman, A. Aly, B. Chen, and C.-J. Wu. Layerskip: Enabling early exit inference and self-speculative decoding. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), page 12622–12642. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.681. URL http://dx.doi.org/10.18653/v1/2024.acl-long.681.

Fireworks AI. Speed, Python: Pick Two. How CUDA Graphs Enable Fast Python Code for Deep Learning. https://fireworks.ai/blog/speed-python-pick-two-how-cuda-graphs-enable-fast-python-code-for-deep-learning, Aug. 2023. Accessed: 2026-06-22.

T. Ge, H. Xia, X. Sun, S.-Q. Chen, and F. Wei. Lossless acceleration for seq2seq generation with aggressive decoding. arXiv preprint arXiv:2205.10350, 2022.

M. Ghazvininejad, O. Levy, Y. Liu, and L. Zettlemoyer. Mask-predict: Parallel decoding of conditional masked language models. In K. Inui, J. Jiang, V. Ng, and X. Wan, editors, Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pages 6112–6121, Hong Kong, China, Nov. 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1633. URL https://aclanthology.org/D19-1633/.

F. Gloeckle, B. Youbi Idrissi, B. Roziere, D. Lopez-Paz, and G. Synnaeve. Better & faster large language models via multi-token prediction. In R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, editors, Proceedings of the 41st International Conference on Machine Learning, volume 235 of Proceedings of Machine Learning Research, pages 15706–15734. PMLR, 21–27 Jul 2024. URL https://proceedings.mlr.press/v235/gloeckle24a.html.

Google DeepMind. Gemma 4 model card. https://ai.google.dev/gemma/docs/core/model_card_4, 2026. Accessed: 2026-06-11.

J. Gu, J. Bradbury, C. Xiong, V. O. Li, and R. Socher. Non-autoregressive neural machine translation. In International Conference on Learning Representations, 2018. URL https://openreview.net/forum?id=B1l8BtlCb.

S. Gui, C. Shao, Z. Ma, X. Zhang, Y. Chen, and Y. Feng. Non-autoregressive machine translation with probabilistic context-free grammar. In Thirty-seventh Conference on Neural Information Processing Systems, 2023. URL https://openreview.net/forum?id=LloZFVwWvj.

C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger. On calibration of modern neural networks. In International conference on machine learning, pages 1321–1330. PMLR, 2017.

J. A. Hanley and B. J. McNeil. The meaning and use of the area under a receiver operating characteristic (roc) curve. Radiology, 143(1):29–36, 1982. doi: 10.1148/radiology.143.1.7063747.

Z. He, Z. Zhong, T. Cai, J. D. Lee, and D. He. Rest: Retrieval-based speculative decoding, 2023.

L. Hu, Z. Feng, Y. Wu, H. Yuan, Y. Zhao, Y.-Y. Qian, B. Wang, P. Zhao, D. Jiang, Y. Zhu, T. Rosing, and H. Zhang. Jetspec: Breaking the scaling ceiling of speculative decoding with parallel tree drafting, 2026a. URL https://arxiv.org/abs/2606.18394.

X. Hu, Y. Shen, B. Zhang, H. Zhang, J. Dai, S. Ge, L. Chen, Y. Li, and M. Wan. Echo: Elastic speculative decoding with sparse gating for high-concurrency scenarios. arXiv preprint arXiv:2604.09603, 2026b.

Y. Hu, K. Wang, X. Zhang, F. Zhang, C. Li, H. Chen, and J. Zhang. SAM decoding: Speculative decoding via suffix automaton. In W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, editors, Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 12187–12204, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.595. URL https://aclanthology.org/2025.acl-long.595/.

F. Huang, T. Tao, H. Zhou, L. Li, and M. Huang. On the learning of non-autoregressive transformers. In K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato, editors, Proceedings of the 39th International Conference on Machine Learning, volume 162 of Proceedings of Machine Learning Research, pages 9356–9376. PMLR, 17–23 Jul 2022a. URL https://proceedings.mlr.press/v162/huang22k.html.

F. Huang, H. Zhou, Y. Liu, H. Li, and M. Huang. Directed acyclic transformer for non-autoregressive machine translation. In K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato, editors, Proceedings of the 39th International Conference on Machine Learning, volume 162 of Proceedings of Machine Learning Research, pages 9410–9428. PMLR, 17–23 Jul 2022b. URL https://proceedings.mlr.press/v162/huang22m.html.

J. Huang, Y. Zhang, Q. Zhang, H. Lin, H. Xu, and L. Zhang. Domino: Decoupling causal modeling from autoregressive drafting in speculative decoding, 2026a. URL https://arxiv.org/abs/2605.29707.

K. Huang, X. Guo, and M. Wang. Specdec++: Boosting speculative decoding via adaptive candidate lengths. arXiv preprint arXiv:2405.19715, 2024.

K. Huang, H. Wu, Z. Shi, H. Zou, M. Yu, and Q. Shi. Adaspec: Adaptive speculative decoding for fast, slo-aware large language model serving. In Proceedings of the 2025 ACM Symposium on Cloud Computing, SoCC '25, page 361–374, New York, NY, USA, 2026b. Association for Computing Machinery. ISBN 9798400722769. doi: 10.1145/3772052.3772239. URL https://doi.org/10.1145/3772052.3772239.

M. Hui, X. Huang, J. C. Salas, Y. Sun, N. Pemberton, X. Song, A. Khetan, and G. Karypis. P-eagle: Parallel-drafting eagle with scalable training, 2026. URL https://arxiv.org/abs/2602.01469.

D. Israel, G. Van den Broeck, and A. Grover. Accelerating diffusion llms via adaptive parallel decoding. Advances in neural information processing systems, 38:52870–52888, 2026.

N. Jain, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. In International Conference on Learning Representations, volume 2025, pages 58791–58831, 2025.

L. Kaiser, S. Bengio, A. Roy, A. Vaswani, N. Parmar, J. Uszkoreit, and N. Shazeer. Fast decoding in sequence models using discrete latent variables. In J. Dy and A. Krause, editors, Proceedings of the 35th International Conference on Machine Learning, volume 80 of Proceedings of Machine Learning Research, pages 2390–2399. PMLR, 10–15 Jul 2018. URL https://proceedings.mlr.press/v80/kaiser18a.html.

W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with pagedattention. In Proceedings of the 29th symposium on operating systems principles, pages 611–626, 2023.

Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, editors, Proceedings of the 40th International Conference on Machine Learning, volume 202 of Proceedings of Machine Learning Research, pages 19274–19286. PMLR, 23–29 Jul 2023. URL https://proceedings.mlr.press/v202/leviathan23a.html.

G. Li, Z. Fu, M. Fang, Q. Zhao, M. Tang, C. Yuan, and J. Wang. Diffuspec: Unlocking diffusion language models for speculative decoding, 2025a. URL https://arxiv.org/abs/2510.02358.

R. Li, Z. Zhang, L. Zhang, H. Wang, X. Fu, and Z. Lai. Nightjar: Dynamic adaptive speculative decoding for large language models serving, 2026a. URL https://arxiv.org/abs/2512.22420.

T. Li, W.-L. Chiang, E. Frick, L. Dunlap, B. Zhu, J. E. Gonzalez, and I. Stoica. From live data to high-quality benchmarks: The arena-hard pipeline, April 2024a. URL https://lmsys.org/blog/2024-04-19-arena-hard/.

T. Li, W.-L. Chiang, E. Frick, L. Dunlap, T. Wu, B. Zhu, J. E. Gonzalez, and I. Stoica. From crowdsourced data to high-quality benchmarks: Arena-hard and benchbuilder pipeline. In A. Singh, M. Fazel, D. Hsu, S. Lacoste-Julien, F. Berkenkamp, T. Maharaj, K. Wagstaff, and J. Zhu, editors, Proceedings of the 42nd International Conference on Machine Learning, volume 267 of Proceedings of Machine Learning Research, pages 34209–34231. PMLR, 13–19 Jul 2025b. URL https://proceedings.mlr.press/v267/li25h.html.

X. L. Li, J. Thickstun, I. Gulrajani, P. Liang, and T. Hashimoto. Diffusion-LM improves controllable text generation. In A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, editors, Advances in Neural Information Processing Systems, 2022. URL https://openreview.net/forum?id=3s9IrEsjLyk.

Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. In Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, editors, Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pages 7421–7432, Miami, Florida, USA, Nov. 2024b. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.422. URL https://aclanthology.org/2024.emnlp-main.422/.

Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, editors, Proceedings of the 41st International Conference on Machine Learning, volume 235 of Proceedings of Machine Learning Research, pages 28935–28948. PMLR, 21–27 Jul 2024c. URL https://proceedings.mlr.press/v235/li24bt.html.

Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. In The Thirty-ninth Annual Conference on Neural Information Processing Systems, 2026b. URL https://openreview.net/forum?id=4exx1hUffq.

J. Libovický and J. Helcl. End-to-end non-autoregressive neural machine translation with connectionist temporal classification. In E. Riloff, D. Chiang, J. Hockenmaier, and J. Tsujii, editors, Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, pages 3016–3021, Brussels, Belgium, Oct.-Nov. 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1336. URL https://aclanthology.org/D18-1336/.

H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let's verify step by step. In The Twelfth International Conference on Learning Representations, 2024. URL https://openreview.net/forum?id=v8L0pN6EOi.

F. Liu, Y. Tang, Z. Liu, Y. Ni, D. Tang, K. Han, and Y. Wang. Kangaroo: Lossless self-speculative decoding for accelerating llms via double early exiting. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, Advances in Neural Information Processing Systems, volume 37, pages 11946–11965. Curran Associates, Inc., 2024a. doi: 10.52202/079017-0381. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/16336d94a5ffca8de019087ab7fe403f-Paper-Conference.pdf.

F. Liu, X. Li, K. Zhao, Y. Gao, Z. Zhou, Z. Zhang, Z. Wang, W. Dou, S. Zhong, and C. Tian. Dart: Diffusion-inspired speculative decoding for fast llm inference, 2026a. URL https://arxiv.org/abs/2601.19278.

H. Liu, J. Huang, Z. Jia, Y. Park, and Y.-X. Wang. Not-a-bandit: Provably no-regret drafter selection in speculative decoding for llms, 2026b. URL https://arxiv.org/abs/2510.20064.

T. Liu, Q. Lv, Y. Shen, X. Sun, and X. Sun. Talon: Confidence-aware speculative decoding with adaptive token trees. arXiv preprint arXiv:2601.07353, 2026c.

X. Liu, C. Daniel, L. Hu, W. Kwon, Z. Li, X. Mo, A. Cheung, Z. Deng, I. Stoica, and H. Zhang. Optimizing speculative decoding for serving large language models using goodput. arXiv e-prints, pages arXiv–2406, 2024b.

X. Liu, J. Park, L. Hu, W. Kwon, Z. Li, C. Zhang, K. Du, X. Mo, K. You, A. Cheung, et al. Turbospec: Closed-loop speculation control system for optimizing llm serving goodput. arXiv preprint arXiv:2406.14066, 2024c.

X. Ma, C. Zhou, X. Li, G. Neubig, and E. Hovy. FlowSeq: Non-autoregressive conditional sequence generation with generative flow. In K. Inui, J. Jiang, V. Ng, and X. Wan, editors, Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pages 4282–4292, Hong Kong, China, Nov. 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1437. URL https://aclanthology.org/D19-1437/.

J. Mamou, O. Pereg, D. Korat, M. Berchansky, N. Timor, M. Wasserblat, and R. Schwartz. Dynamic speculation lookahead accelerates speculative decoding of large language models. In M. Rezagholizadeh, P. Passban, S. Samiee, V. Partovi Nia, Y. Cheng, Y. Deng, Q. Liu, and B. Chen, editors, Proceedings of The 4th NeurIPS Efficient Natural Language and Speech Processing Workshop, volume 262 of Proceedings of Machine Learning Research, pages 456–467. PMLR, 14 Dec 2024. URL https://proceedings.mlr.press/v262/mamou24a.html.

X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi, et al. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, pages 932–949, 2024.

M. P. Naeini, G. Cooper, and M. Hauskrecht. Obtaining well calibrated probabilities using bayesian binning. In Proceedings of the AAAI conference on artificial intelligence, volume 29, 2015.

Y. Ovadia, E. Fertig, J. Ren, Z. Nado, D. Sculley, S. Nowozin, J. Dillon, B. Lakshminarayanan, and J. Snoek. Can you trust your model's uncertainty? evaluating predictive uncertainty under dataset shift. Advances in neural information processing systems, 32, 2019.

L. Qian, H. Zhou, Y. Bao, M. Wang, L. Qiu, W. Zhang, Y. Yu, and L. Li. Glancing transformer for non-autoregressive neural machine translation. In C. Zong, F. Xia, W. Li, and R. Navigli, editors, Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers), pages 1993–2003, Online, Aug. 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-long.155. URL https://aclanthology.org/2021.acl-long.155/.

Y. Ren, J. Liu, X. Tan, Z. Zhao, S. Zhao, and T.-Y. Liu. A study of non-autoregressive model for sequence generation. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pages 149–159, 2020.

L. Ringel and Y. Romano. Accelerating speculative decoding with block diffusion draft trees. arXiv preprint arXiv:2604.12989, 2026.

R. Sadhukhan, J. Chen, Z. Chen, V. Tiwari, R. Lai, J. Shi, I. E.-H. Yen, A. May, T. Chen, and B. Chen. Magicdec: Breaking the latency-throughput tradeoff for long context generation with speculative decoding. In The Thirteenth International Conference on Learning Representations, 2025. URL https://openreview.net/forum?id=CS2JWaziYr.

C. Saharia, W. Chan, S. Saxena, and M. Norouzi. Non-autoregressive machine translation with latent alignments. In B. Webber, T. Cohn, Y. He, and Y. Liu, editors, Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), pages 1098–1108, Online, Nov. 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-main.83. URL https://aclanthology.org/2020.emnlp-main.83/.

J. Sandler, J. Christopher, T. Hartvigsen, and F. Fioretto. Specdiff-2: Scaling diffusion drafter alignment for faster speculative decoding. In Ninth Conference on Machine Learning and Systems, 2026. URL https://openreview.net/forum?id=o42VU86ZsV.

A. Saxena. Prompt lookup decoding, November 2023. URL https://github.com/apoorvumang/prompt-lookup-decoding/.

C. Shao, Y. Feng, J. Zhang, F. Meng, and J. Zhou. Sequence-level training for non-autoregressive neural machine translation. Computational Linguistics, 47(4):891–925, Dec. 2021. doi: 10.1162/coli_a_00421. URL https://aclanthology.org/2021.cl-4.29/.

C. Shao, Z. Ma, M. Zhang, and Y. Feng. Beyond MLE: Convex learning for text generation. In Thirty-seventh Conference on Neural Information Processing Systems, 2023. URL https://openreview.net/forum?id=sla7V80uWA.

Y. Shen, T. Liu, X. Hu, Q. Kong, B. Zhang, J. Dai, J. Zhang, S. Ge, L. Chen, Y. Li, M. Wan, and C. Wang. Draft less, retrieve more: Hybrid tree construction for speculative decoding, 2026. URL https://arxiv.org/abs/2605.20104.

S. Somasundaram, A. Phukan, and A. Saxena. PLD+: Accelerating LLM inference by leveraging language model artifacts. In L. Chiruzzo, A. Ritter, and L. Wang, editors, Findings of the Association for Computational Linguistics: NAACL 2025, pages 6090–6104, Albuquerque, New Mexico, Apr. 2025. Association for Computational Linguistics. ISBN 979-8-89176-195-7. doi: 10.18653/v1/2025.findings-naacl.338. URL https://aclanthology.org/2025.findings-naacl.338/.

M. Stern, N. Shazeer, and J. Uszkoreit. Blockwise parallel decoding for deep autoregressive models. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, Advances in Neural Information Processing Systems, volume 31. Curran Associates, Inc., 2018. URL https://proceedings.neurips.cc/paper_files/paper/2018/file/c4127b9194fe8562c64dc0f5bf2c93bc-Paper.pdf.

X. Sun, T. Ge, F. Wei, and H. Wang. Instantaneous grammatical error correction with shallow aggressive decoding. In C. Zong, F. Xia, W. Li, and R. Navigli, editors, Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers), pages 5937–5947, Online, Aug. 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-long.462. URL https://aclanthology.org/2021.acl-long.462/.

Z. Sun, Z. Li, H. Wang, D. He, Z. Lin, and Z. Deng. Fast structured decoding for sequence models. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, Advances in Neural Information Processing Systems, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/74563ba21a90da13dacf2a73e3ddefa7-Paper.pdf.

R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca, 2023.

S. Tiwari, T. Chugh, N. Rickert, S. Peter, R. Mahajan, and H. Shen. Cachewise: Understanding workloads and optimizing kvcache management for efficiently serving llm coding agents. arXiv preprint arXiv:2606.16824, 2026.

C. Wang, J. Zhang, and H. Chen. Semi-autoregressive neural machine translation. In E. Riloff, D. Chiang, J. Hockenmaier, and J. Tsujii, editors, Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, pages 479–488, Brussels, Belgium, Oct.-Nov. 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1044. URL https://aclanthology.org/D18-1044/.

Z. Wang, J. Huang, and X. Chen. Taps: Target-aware prefix tree selection for diffusion-drafted speculative decoding, 2026a. URL https://arxiv.org/abs/2606.00487.

Z. Wang, D. Ma, X. Huang, D. Cai, T. Lan, J. Xu, H. Mi, X. Tang, and Y. Wang. THE END OF MANUAL DECODING: TOWARDS TRULY END-TO-END LANGUAGE MODELS. In The Fourteenth International Conference on Learning Representations, 2026b. URL https://openreview.net/forum?id=cPTgQDMD5p.

Z. Wen and Y. Feng. Specbound: Adaptive bounded self-speculation with layer-wise confidence calibration, 2026. URL https://arxiv.org/abs/2604.12247.

Z. Wen, S. Gui, and Y. Feng. Speculative decoding with ctc-based draft model for llm inference acceleration. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, Advances in Neural Information Processing Systems, volume 37, pages 92082–92100. Curran Associates, Inc., 2024. doi: 10.52202/079017-2923. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/a79054a9da91d73ed3cb1a9e87d7cd2d-Paper-Conference.pdf.

M. Williams, Y. D. Kwon, R. Li, A. Kouris, and S. I. Venieris. Speculative decoding with a speculative vocabulary. arXiv preprint arXiv:2602.13836, 2026.

Z. Wu, Z. Zhou, A. Verma, A. Prakash, D. Rus, and B. K. H. Low. TETRIS: Optimal draft token selection for batch speculative decoding. In W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, editors, Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 33329–33345, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.1598. URL https://aclanthology.org/2025.acl-long.1598/.

H. Xia, T. Ge, P. Wang, S.-Q. Chen, F. Wei, and Z. Sui. Speculative decoding: Exploiting speculative execution for accelerating seq2seq generation. In H. Bouamor, J. Pino, and K. Bali, editors, Findings of the Association for Computational Linguistics: EMNLP 2023, pages 3909–3925, Singapore, Dec. 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.257. URL https://aclanthology.org/2023.findings-emnlp.257/.

H. Xia, Z. Yang, Q. Dong, P. Wang, Y. Li, T. Ge, T. Liu, W. Li, and Z. Sui. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. In L.-W. Ku, A. Martins, and V. Srikumar, editors, Findings of the Association for Computational Linguistics ACL 2024, pages 7655–7671, Bangkok, Thailand and virtual meeting, Aug. 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.456. URL https://aclanthology.org/2024.findings-acl.456.

H. Xia, Y. Li, J. Zhang, C. Du, and W. Li. SWIFT: On-the-fly self-speculative decoding for LLM inference acceleration. In The Thirteenth International Conference on Learning Representations, 2025. URL https://openreview.net/forum?id=EKJhH5D5wA.

Z. Xie, Y. Wei, H. Cao, C. Zhao, C. Deng, J. Li, D. Dai, H. Gao, M. Xu, K. Yu, L. Zhao, S. Zhou, Z. Xu, Z. Zhang, W. Zeng, S. Hu, Y. Wang, J. Yuan, L. Wang, and W. Liang. mHC: Manifold-constrained hyper-connections. In Forty-third International Conference on Machine Learning, 2026. URL https://openreview.net/forum?id=mDhyxu8WRb.

T. Xu, E. Helenowski, K. A. Sankararaman, D. Jin, K. Peng, E. Han, S. Nie, C. Zhu, H. Zhang, W. Zhou, et al. The perfect blend: Redefining rlhf with mixture of judges. arXiv preprint arXiv:2409.20370, 2024.

D. Yan, W. Wang, and X. Chu. Demystifying tensor cores to optimize half-precision matrix multiply. 2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS), pages 634–643, 2020. URL https://api.semanticscholar.org/CorpusID:220604999.

A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, et al. Qwen3 technical report. arXiv preprint arXiv:2505.09388, 2025.

Zacks917. AutoMTP_vLLM: Adapt vllm to automtp (early stop for multi-token prediction). https://github.com/Zacks917/AutoMTP_vLLM, 2026. Accessed: 2026-06-21.

J. Zhang, J. Wang, H. Li, L. Shou, K. Chen, G. Chen, and S. Mehrotra. Draft& verify: Lossless large language model acceleration via self-speculative decoding. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), page 11263–11282. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.607. URL http://dx.doi.org/10.18653/v1/2024.acl-long.607.

J. Zhang, Z. Yu, S. Liu, E. J. Yu, Z. Li, D. Zhu, J. Duo, W. Xiong, Y. Song, G. Yu, J. Zhu, and S. Li. Dflare: Scaling up draft capacity for block diffusion speculative decoding, 2026a. URL https://arxiv.org/abs/2606.02091.

L. Zhang, X. Wang, Y. Huang, and R. Xu. Learning harmonized representations for speculative sampling, 2025. URL https://arxiv.org/abs/2408.15766.

S. Zhang, Y. Zhang, Z. Zhu, H. Wang, D. Ma, D. Zhang, L. Chen, and K. Yu. Pacer: Blockwise pre-verification for speculative decoding with adaptive length. arXiv preprint arXiv:2602.01274, 2026b.

Y. Zhang and T. Math-AI. American invitational mathematics examination (aime) 2025, 2025.

C. Zhao, C. Deng, C. Ruan, D. Dai, H. Gao, J. Li, L. Zhang, P. Huang, S. Zhou, S. Ma, et al. Insights into deepseek-v3: Scaling challenges and reflections on hardware for ai architectures. In Proceedings of the 52nd Annual International Symposium on Computer Architecture, pages 1731–1745, 2025a.

W. Zhao, T. Pan, X. Han, Y. Zhang, S. Ao, Y. Huang, K. Zhang, W. Zhao, Y. Li, J. Zhou, et al. Fr-spec: Accelerating large-vocabulary language models via frequency-ranked speculative sampling. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 3909–3921, 2025b.

K. Zheng, Y. Chen, H. Mao, M.-Y. Liu, J. Zhu, and Q. Zhang. Masked diffusion models are secretly time-agnostic masked models and exploit inaccurate categorical sampling. In The Thirteenth International Conference on Learning Representations, 2025. URL https://openreview.net/forum?id=CTC7CmirNr.

L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. Advances in neural information processing systems, 36:46595–46623, 2023.

L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng. Sglang: Efficient execution of structured language model programs. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, Advances in Neural Information Processing Systems, volume 37, pages 62557–62583. Curran Associates, Inc., 2024. doi: 10.52202/079017-2000. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/724be4472168f31ba1c9ac630f15dec8-Paper-Conference.pdf.

Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), pages 193–210, 2024.

K. Zhu, Y. Gao, Y. Zhao, L. Zhao, G. Zuo, Y. Gu, D. Xie, T. Tang, Q. Xu, Z. Ye, K. Kamahori, C.-Y. Lin, Z. Wang, S. Wang, A. Krishnamurthy, and B. Kasikci. Nanoflow: towards optimal large language model serving throughput. In Proceedings of the 19th USENIX Conference on Operating Systems Design and Implementation, OSDI '25, USA, 2025. USENIX Association. ISBN 978-1-939133-47-2.

Appendices

A. Counterexample: Selection Bias Without Early-Stopping

We provide a simple counterexample to illustrate how an offline global search, i.e., operating without the `break` condition in Algorithm 1, violates the non-anticipating property required by lossless speculative decoding. Formally, the admission event for the k-th draft token, $\ell_r \geq k$, must be determined by scheduler-visible information available before the token $x_{r,k}$ is sampled. It must not depend on the realization of $x_{r,k}$ itself. Consider a scenario with a single request ($R = 1$) and maximum draft length ($\gamma = 2$). Suppose the pre-token confidence for the first position is $a_1 = 0.8$, and the profiled capacity curve is

$$\mathrm{SPS}(1) = 1.0, \qquad \mathrm{SPS}(2) = 0.5, \qquad \mathrm{SPS}(3) = 0.45.$$

The expected throughputs for verifying 0 and 1 draft tokens are

$$\Theta_0 = 1 \cdot \mathrm{SPS}(1) = 1.0,$$
$$\Theta_1 = (1 + 0.8) \cdot \mathrm{SPS}(2) = 0.9.$$

Without early-stopping, the scheduler proceeds to evaluate Θ_2 before committing any admission decisions. Because the Markov confidence head uses the previously sampled token, the next confidence score c_2 explicitly depends on the realization of x_1. Consequently, the second-prefix survival probability

$$a_2 = a_1 c_2$$

also depends on x_1. Consider two possible realizations of x_1:

- **Case 1 (x_1 yields a high c_2):** Suppose x_1 results in $c_2 = 0.9$. Then

  $$a_2 = 0.8 \times 0.9 = 0.72.$$

  The expected throughput for length 2 is

  $$\Theta_2 = (1 + 0.8 + 0.72) \times 0.45 = 1.134.$$

  Since Θ_2 is the global maximum among $\{1.0, 0.9, 1.134\}$, the scheduler returns $\ell = 2$. The first token x_1 is admitted into the verification prefix.
- **Case 2 (x_1 yields a low c_2):** Suppose x_1 results in $c_2 = 0$. Then

  $$a_2 = 0.$$

  The expected throughput for length 2 is

  $$\Theta_2 = (1 + 0.8 + 0) \times 0.45 = 0.81.$$

  Here, the global maximum remains $\Theta_0 = 1.0$, so the scheduler returns $\ell = 0$. The first token x_1 is not admitted into the verification prefix.

Thus, the admission of the first draft token dynamically depends on the value of the first draft token itself. This retrospective dependence introduces selection bias: the scheduler favors tokens that lead to highly confident continuations, even though the admission decision for x_1 should have been made before observing x_1. We now make the distributional bias explicit. Let the vocabulary be $\{A, B\}$, and consider the target and draft distributions at the first position:

$$p_{\mathrm{t}}(A) = 0.7, \qquad p_{\mathrm{t}}(B) = 0.3,$$

$$p_{\mathrm{d}}(A) = 0.5, \qquad p_{\mathrm{d}}(B) = 0.5.$$

The standard speculative acceptance probability at the first position is

$$\sum_{x \in \{A,B\}} \min\big(p_{\mathrm{t}}(x), p_{\mathrm{d}}(x)\big) = \min(0.7, 0.5) + \min(0.3, 0.5) = 0.8,$$

matching the assumed value ($a_1 = 0.8$). Suppose the retrospective scheduler behaves as above: $x_1 = A$ yields a high continuation confidence and hence $\ell = 2$, while $x_1 = B$ yields a low continuation confidence and hence $\ell = 0$. Then the first output token is distributed as follows. If $x_1 = A$, the draft token is admitted and accepted with probability

$$\min\left(1, \frac{p_{\mathrm{t}}(A)}{p_{\mathrm{d}}(A)}\right) = \min\left(1, \frac{0.7}{0.5}\right) = 1,$$

so the output token is A. If $x_1 = B$, the draft token is not admitted; the target model instead generates a fresh token from p_{t}. Therefore,

$$\Pr(Y = A) = \Pr(x_1 = A) \cdot 1 + \Pr(x_1 = B) \cdot p_{\mathrm{t}}(A) = 0.5 + 0.5 \times 0.7 = 0.85,$$

and hence

$$\Pr(Y = B) = 0.15.$$

This output distribution ($(0.85, 0.15)$) differs from the target distribution ($(0.7, 0.3)$), proving that the retrospective scheduler is not lossless. The early-stopping mechanism prevents this issue in the causal greedy scheduler. Since $\Theta_1 < \Theta_0$, the scheduler halts immediately and returns $\ell = 0$ before evaluating any continuation-dependent quantity such as c_2. The admission decision for the first position therefore depends only on pre-token information and cannot be biased by the realization of x_1. This restores the non-anticipating property required by the standard losslessness argument.